ARCHITECTURE & ALGORITHMS · READER EDITION

Claude Code 시스템 해부

복원 소스에서 실행 경계·상태·효과·복구·확장 구조를 읽는
아키텍처와 알고리즘

한국어 독자판 · 고정 소스 revision 2ca5ddab

이 책은 알고리즘을 증명하거나 키워드로 분류하는 대신, 왜 그 코드가 필요하며 어떤 언어 메커니즘·상태·소유권·실패 경계를 함께 읽어야 현재 시스템을 재구성할 수 있는지 설명한다.

읽기 계약

증거와 출처 경계

코드가 존재한다는 사실, 구조에 대한 해석, 개선 제안, 현재 자료로 알 수 없는 것을 같은 문장으로 섞지 않는다.

OBSERVED

분석 대상은 배포 bundle을 TypeScript 모듈로 복원한 고정 snapshot이며, 파일·타입·분기·호출은 이 snapshot 안에서 관찰한다.

INFERENCE

여러 파일의 관계에서 도출한 설계 해석은 가능한 설명이지 upstream 작성자의 유일한 의도나 production 활성 상태의 증명이 아니다.

RECOMMENDATION

리팩터링과 검증 제안은 현재 구조가 노출하는 위험을 줄이기 위한 독자판의 판단이며 원 저장소의 계획으로 돌려 말하지 않는다.

UNKNOWN

공식 모노레포 경계, 배포별 feature 값, server protocol, telemetry backend, 실제 외부 effect settlement는 이 자료만으로 닫지 않는다.

연구와 설명 방법

분석 방법

먼저 독자가 답하고 싶은 시스템 질문을 고른다. 그런 다음 정의와 참조, 소유 파일, 상태 전이, effect 경계를 거슬러 올라가며 코드 조각이 맡는 책임을 좁힌다.

코드는 단독으로 인용하지 않는다. 필요한 선행 개념과 축소 예제를 먼저 세우고, 실제 source range로 돌아와 그 개념이 어떤 보장과 실패 가능성을 만든다고 읽을 수 있는지 연결한다.

현재 상태는 OBSERVED·INFERENCE·RECOMMENDATION·UNKNOWN으로 분리한다. 실행 순서는 인과관계를 설명할 때만 사용하고, 시스템 전체를 순서도 하나로 환원하지 않는다.

본문은 사례 연구와 안내식 코드 독해를 담당하고, 알고리즘·source·용어·요구사항·연구 채택표는 부록에서 빠르게 찾을 수 있는 참조 체계로 제공한다.

외부 방법 연구와 채택·기각 기록 보기

한 페이지 기준선

현재 상태 대시보드

OBSERVED

- CLI·headless·remote·MCP 등 여러 진입 표면이 명시적 dispatch와 조합 경계를 가진다.
- query/stream/tool/transcript 계층은 turn 상태와 effect 결과를 서로 다른 표현으로 다룬다.
- permission, hook, sandbox, file commit, recovery, extension 경계가 독립된 결정 지점을 형성한다.

INFERENCE

- 이 구조의 핵심 난제는 기능 수가 아니라 한 turn의 canonical state와 외부 effect 사이의 시간차를 관리하는 일이다.
- 동적 import, tagged union, async generator, append-only transcript는 각각 다른 위치에서 불완전한 상태의 전파를 제한한다.

RECOMMENDATIONS

- route precedence, state transition, permission/effect commit, stream reconstruction을 characterization/property test의 우선 대상으로 삼는다.
- 관찰 event와 canonical state, UI projection을 이름과 타입에서 구분해 retry와 resume 경계를 드러낸다.

UNKNOWNS

- production에서 활성인 feature 조합과 각 경로의 사용 빈도
- 복원 과정에서 누락·변형된 모듈과 공식 테스트·배포 이력
- 서버 측 세부 protocol, telemetry 저장·처리, 실제 외부 effect 정산 상태

계층적 CONTEXT

전체 시스템 지도

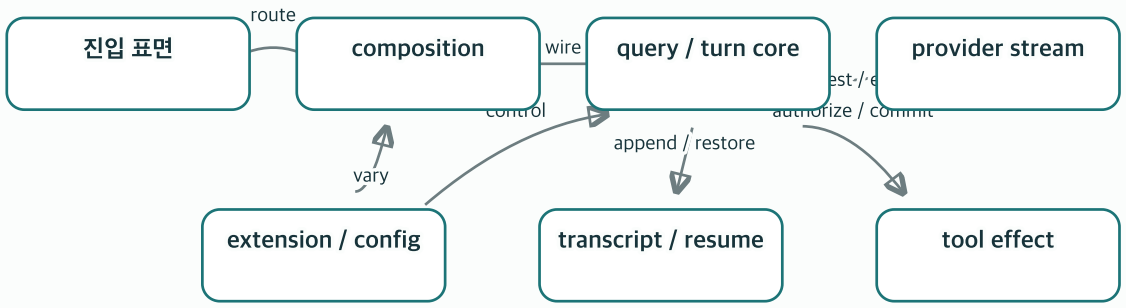


그림 1
전체 시스템 토폴로지 — 진입 표면은 composition을 통해 query core를 선택하고, core는 stream과 tool effect를 조율한다. transcript와 extension은 이 흐름의 옆에서 각각 지속성과 변형 가능성을 소유한다.

이 지도는 배포 topology를 증명하지 않는다. 고정 source에서 관찰한 책임 관계를 이후 Part에서 더 작은 building block과 runtime decision으로 확대하기 위한 독해 좌표다.

스터디 경로

10분·60분·심층 읽기

10분

무엇이 관찰되었고 무엇이 아직 미지인지, 시스템의 여섯 책임 경계를 한 번에 잡는다.

1. 현재 상태 대시보드
2. 전체 시스템 지도
3. 결론

60분

process 진입부터 turn·stream·effect·durability로 이어지는 핵심 상태 경계를 실제 코드와 함께 읽는다.

1. Part I
2. A10 queryLoop
3. A39 stream 재구성
4. A23 file commit
5. 요구사항 원장

심층 스터디

새 기능이나 리팩터링이 어느 소유권·보장·운영 경계를 바꾸는지 설명하고 검토 질문을 만들 수 있다.

1. 모든 본문
2. 10개 심층 독해
3. A01-A44 atlas
4. source index
5. 연구 채택 추적

탐색

목차

1. 증거와 출처 경계
2. 분석 방법
3. 현재 상태 대시보드
4. 전체 시스템 지도
5. 읽기 경로
6. Part I. 프로세스는 어디에서 시작되고 누가 상태를 소유하는가
    1. 복원 소스를 읽기 위한 기준선
    2. 진입점, composition, runtime surface
7. Part II. 입력에서 streaming까지, 한 turn은 어떻게 완성되는가
    1. 입력과 context는 message가 되기 전부터 구조를 갖는다
    2. query state machine과 typed stream 재구성
8. Part III. 도구, 동시성, 권한, 파일 변경은 어떻게 effect를 제한하는가
    1. 도구 선택과 동시 실행의 보존 조건
    2. permission, hook, sandbox는 하나의 yes/no가 아니다
    3. 파일 변경은 계산 결과가 아니라 commit protocol이다
9. Part IV. transcript, compaction, recovery, nested agent
    1. append-only transcript와 compaction
    2. 오류, 중단, resume의 재구성
    3. nested agent와 독립된 lifecycle
10. Part V. MCP, plugin, remote, config와 운영의 변형 경계
    1. MCP와 plugin registry
    2. remote와 bridge의 transport 경계
    3. configuration과 operations를 읽는 시간축
11. 결론
12. 부록 A · A01-A44 알고리즘 atlas
13. 부록 B · 14개 repository domain
14. 부록 C · source index
15. 부록 D · 용어집
16. 부록 E · 요구사항 원장
17. 부록 F · 연구 채택 추적

I

프로세스는 어디에서 시작되고 누가 상태를 소유하는가

파일 목록이 아니라 route precedence, composition, runtime surface, state ownership을 따라 프로세스의 책임 경계를 세운다.

목차로

장

복원 소스를 읽기 위한 기준선

풍부한 파일 구조와 공식 개발 원본이라는 주장은 다르므로, provenance와 현재 확인 가능한 범위를 먼저 고정해야 한다.

목차

분석의 첫 단계는 코드를 많이 읽는 일이 아니라 어떤 산출물을 읽고 있는지 결정하는 일이다. 이 저장소의 README와 source profile은 배포 bundle에서 풀어낸 연구용 projection임을 말한다. 그러므로 함수와 타입의 존재는 OBSERVED로 기록할 수 있지만, upstream 팀의 의도나 production 활성 여부는 별도 증거 없이 승격하지 않는다.

A01은 출처를 고정하고 A02는 import graph를 구조화한다. 이 두 알고리즘은 제품 기능이 아니지만 뒤의 모든 설명이 과장되지 않도록 만든다. graph의 중심성은 책임의 단서일 뿐 권위나 중요도의 자동 증명이 아니라는 제한도 함께 유지한다.

이 장의 결정점

> **A01 · 복사본 변환과 반복 stub 생성**
>
> 복원 builder는 원본을 실행 가능하게 증명하는 compiler가 아니라 실패를 줄여 가는 best-effort reconstruction loop다.
>
> `scripts/build.mjs:52–117`
>
> **보존 조건.** 원본 src는 build.mjs 경로에서 수정하지 않는다.

> **A02 · 원본을 직접 고치는 prepare-src**
>
> 별도 build 경로와 달리 prepare-src는 source tree 자체를 rewrite한다.
>
> `scripts/prepare-src.mjs:36–76`
>
> **보존 조건.** 재실행은 대부분 동일 결과를 내야 한다.

장

진입점, composition, runtime surface

하나의 binary가 여러 실행 표면을 제공할 때 branch 순서와 dynamic import가 process lifetime과 초기화 비용을 함께 결정한다.

목차

CLI의 if/return 사슬은 사소한 옵션 파서가 아니다. 먼저 참이 된 predicate가 뒤 경로를 배제하고, branch 안의 dynamic import가 선택되지 않은 module의 평가를 늦춘다. 이 때문에 route 순서 변경은 리팩터링처럼 보여도 실제 process owner와 bundle graph를 바꾸는 동작 변경이다.

composition은 이 결정을 각 subsystem의 내부 로직과 분리한다. 표면은 turn을 시작할 수 있지만 query 내부 상태를 소유하지 않고, provider는 event를 만들지만 canonical message의 의미를 소유하지 않는다. 이 소유권 분리를 잡아야 뒤 장의 UI와 streaming을 중복 구현으로 오해하지 않는다.

그림 2 · 상태 소유권과 projection에서 이 장의 정적 책임과 동적 연결을 먼저 확인한다.

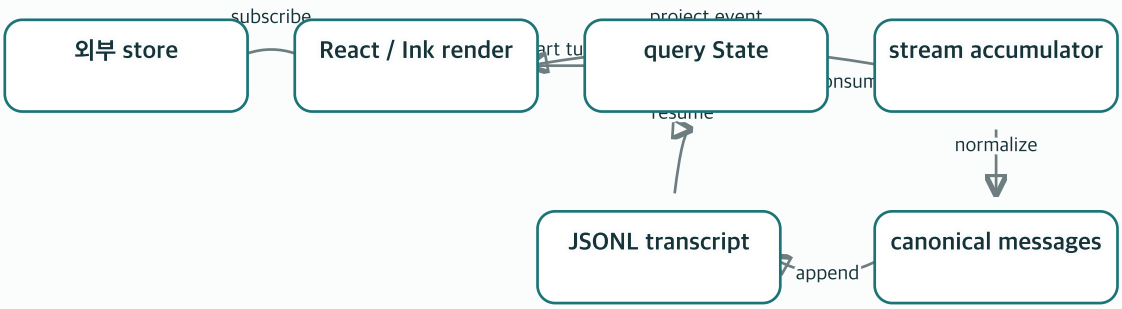


그림 2
상태 소유권과 projection — 외부 store, render state, turn state, canonical transcript는 같은 데이터를 복제한 것이 아니라 서로 다른 결정 시점과 수명을 가진 projection이다.

이 장의 결정점

A03 · 우선순위 기반 CLI route resolution

entrypoint는 ordered early-return dispatcher다.

`src/entrypoints/cli.tsx:28–93`

보존 조건. 하나의 실행에서 정확히 한 top-level route만 선택된다.

A04 · ask()의 one-shot engine adapter

호환 wrapper는 외부 mutable state를 새 QueryEngine에 주입하고 결과 state를 다시 꺼낸다.

`src/QueryEngine.ts:1249–1295`

보존 조건. 한 ask 호출에는 한 engine instance가 있다.

A05 · print loop의 turn 간 상태 handoff

print는 engine을 유지하지 않고 mutableMessages와 readFileState를 loop 바깥에서 보존한다.

`src/cli/print.ts:2140–2188`

보존 조건. 새 turn은 이전 message history를 본다.

> **A06 · state lifetime classification**
>
> 리팩터링 전 각 state를 process·session·turn·tool-call·UI-frame 수명으로 분류한다.
>
> `src/QueryEngine.ts:184–207`
>
> **보존 조건.** 모든 mutable field에는 한 primary owner가 있다.

심층 코드 독해 · A03

우선순위 기반 CLI route resolution

하나의 binary가 version 출력부터 remote·daemon·일반 CLI까지 여러 실행 표면을 제공한다. 사용자는 같은 명령을 실행하지만 실제로는 첫 predicate가 선택한 서로 다른 module graph로 들어간다.

> **독자 질문.** route 순서가 왜 제품 의미와 startup cost를 동시에 결정하며, dynamic import가 그 결정을 어떻게 보존하는가?

이 장의 시작 · A03 ATLAS · 목차

결정적인 코드를 읽기 전에

short-circuit chain과 lazy module loading

앞 predicate가 참이면 뒤 predicate는 평가·실행되지 않는다. dynamic import는 선택되지 않은 route의 module initialization도 늦춰 startup과 dead-code elimination 경계를 만든다.

> **source에서 볼 지점.** 28-93행에서 predicate 순서, early return, import 위치를 한 묶음으로 읽어야 한다.

compile-time gate와 runtime policy의 차이

bundle에서 코드를 제거하는 literal과 실행 중 허용 여부를 판단하는 policy는 서로 다른 시간축이다.

> **source에서 볼 지점.** feature 값이 false일 때 route가 단순 비활성인지, module graph에서 제거 가능한지 구분한다.

겹치는 predicate가 있을 때 첫 번째 route가 process를 소유한다

두 조건이 동시에 참일 수 있는 작은 dispatcher를 통해 순서가 단순 표현이 아니라 precedence임을 본다.

A03 축소 예제

```
async function dispatch(args) {
  if (args.includes('--version')) return 'version'
  if (args.includes('--remote')) return await import('./remote.js')
  return await import('./interactive.js')
}

dispatch(['--version', '--remote']) // 'version'
```

지점	관찰	해석
입력 해석	두 flag가 모두 존재한다.	registry의 두 항목이 아니라 첫 predicate가 authority를 가진다.
첫 branch	version이 terminal result를 반환한다.	remote module은 load되지 않는다.
순서 변경	remote predicate를 앞으로 옮기면 결과와 module graph가 바뀐다.	리팩터링은 동작 변경으로 검토해야 한다.

> **프로젝트로 돌아가기.** 실제 dispatcher에서도 fast path의 순서와 각 branch의 return을 characterization test로 고정해야 한다.

고정 source에서 결정점을 찾는다

하나의 binary에서 version, MCP host, remote, daemon, background, 일반 CLI를 충돌 없이 선택한다.

src/entrypoints/cli.tsx:28-93 · fast-path dispatcher

```
 28  /**
 29   * Bootstrap entrypoint - checks for special flags before ↩
loading the full CLI.
 30   * All imports are dynamic to minimize module evaluation for ↩
fast paths.
 31   * Fast-path for --version has zero imports beyond this file.
 32   */
 33  async function main(): Promise<void> {
 34    const args = process.argv.slice(2);
 35
 36    // Fast-path for --version/-v: zero module loading needed
 37    if (args.length === 1 && (args[0] === '--version' || args[0] ↩
=== '-v' || args[0] === '-V')) {
 38      // MACRO.VERSION is inlined at build time
 39      // biome-ignore lint/suspicious/noConsole:: intentional ↩
console output
 40      console.log(`${MACRO.VERSION} (Claude Code)`);
 41      return;
 42    }
 43
 44    // For all other paths, load the startup profiler
 45    const {
 46      profileCheckpoint
 47    } = await import('../utils/startupProfiler.js');
 48    profileCheckpoint('cli_entry');
 49
 50    // Fast-path for --dump-system-prompt: output the rendered ↩
system prompt and exit.
 51    // Used by prompt sensitivity evals to extract the system ↩
prompt at a specific commit.
 52    // Ant-only: eliminated from external builds via feature flag.
 53    if (feature('DUMP_SYSTEM_PROMPT') && args[0] === ↩
'--dump-system-prompt') {
 54      profileCheckpoint('cli_dump_system_prompt_path');
 55      const {
 56        enableConfigs
 57      } = await import('../utils/config.js');
     … 36개 행은 source link에서 이어 읽기
```

고정 revision의 전체 범위 열기

구조 안에서 누가 소유하는가

OBSERVED	src/entrypoints/cli.tsx의 top-level dispatcher가 argv와 compile-time gate를 읽어 오직 하나의 route를 선택한다.
INFERENCE	entrypoint는 각 subsystem의 동작을 소유하지 않고, 어느 subsystem이 process lifetime을 소유할지를 먼저 결정한다.

입력. process.argv · compile-time feature 값 · 환경 변수

유지 state. 아직 load되지 않은 module graph · startup profiler checkpoint

상태가 어떻게 바뀌는가

1. 가장 값싼 --version을 먼저 검사한다.
2. 특수 process flag를 고정 순서로 검사한다.
3. 필요한 path에서만 dynamic import한다.
4. 일치하지 않으면 일반 application bootstrap으로 이동한다.

registry보다 명시적 early return을 택해 startup cost와 DCE 가능성을 우선한다.

보존 조건과 깨지는 방식

- 하나의 실행에서 정확히 한 top-level route만 선택된다.
- fast path는 불필요한 module side effect를 평가하지 않는다.
- feature-excluded path는 외부 bundle에서 사라질 수 있다.

반증 가능한 실패

- predicate 순서 충돌
- top-level import의 조기 side effect
- gate와 runtime policy의 불일치

배경 이론은 source로 돌아오기 위한 도구다

- **chain of responsibility** — 앞선 handler가 처리 여부를 결정하고 다음 후보로 넘길지를 선택하는 책임 사슬이다.
- **lazy loading** — 필요해진 시점까지 module 평가와 자원 획득을 미루는 메커니즘이다.
- **partial evaluation** — 고정된 feature·환경 값을 build 시점에 적용해 남는 프로그램을 좁히는 관점이다.

route table을 문서화하되 dynamic import와 compile-time literal을 훼손하지 않는 characterization test를 둔다.

실행과 운영에서 생기는 결과

OBSERVED	선택된 route만 자신의 초기화와 event loop lifetime을 시작한다.
INFERENCE	top-level import가 생기면 선택되지 않은 표면도 초기화 side effect를 만들 수 있다.

UNKNOWN	겹치는 predicate나 gate/policy 불일치는 잘못된 owner를 선택하거나 사용 불가능한 route를 노출한다.
RECOMMENDATION	route name, 선택 predicate, startup checkpoint를 함께 기록해야 느린 초기화와 잘못된 dispatch를 구분할 수 있다.

지금 재구성할 수 있는 시스템 상태

OBSERVED	고정 source에는 명시적 순서의 early return과 route별 dynamic import가 있다.
INFERENCE	compile-time feature 값과 argv 조합에 따라 일부 route만 후보가 된다.
UNKNOWN	배포 variant별 literal 값과 실제 사용 빈도는 snapshot으로 확정할 수 없다.
RECOMMENDATION	새 route 추가 시 앞뒤 predicate의 교집합과 선택되지 않은 module의 top-level effect를 먼저 검사해야 한다.

다음 코드를 읽을 때 가져갈 질문

- 새 --remote-dev flag가 기존 remote predicate에도 맞는다면 어느 route가 선택되며 그 근거는 어디에 있는가?
- dynamic import를 static import로 바꾸면 startup과 bundle variant에서 어떤 보장이 사라지는가?

II

입력에서 streaming까지, 한 turn은 어떻게 완성되는가

입력·context·message·turn state·partial event가 언제 관찰값이고 언제 canonical state가 되는지 구분한다.

목차로

장

입력과 context는 message가 되기 전부터 구조를 갖는다

사용자 문자열은 history, system context, attachment, policy와 결합되며 이 조합 순서가 model이 볼 수 있는 현재를 만든다.

목차

입력 처리의 결정적 질문은 문자열을 어디에 붙이는가가 아니다. 어느 값이 한 turn 동안 고정되고, 어느 값이 provider 호출마다 다시 계산되며, 어느 값이 transcript에 남는지 묻는 일이다. context budget과 attachment 정규화는 표현 형식을 바꾸면서도 출처와 identity를 잃지 않아야 한다.

UI의 외부 store 구독, ref와 render state, 입력 propagation은 이 과정의 한 관찰창이다. 그러나 UI가 전체 시스템을 대표하지는 않는다. UI는 turn event를 즉시 projection하고 입력 우선순위를 조정하지만, canonical message와 effect 정산은 query·tool·transcript 계층의 책임으로 남는다.

이 장의 결정점

> **A07 · 사용자 입력 dispatch**
>
> 입력은 slash command, bash mode, image, normal prompt를 구별하는 작은 language parser다.
>
> `src/utils/processUserInput/processUserInput.ts:515–588`
>
> **보존 조건.** local-only output을 provider input과 구분한다.

> **A08 · System/User/System context 조립**
>
> prompt는 정적 문자열이 아니라 ordered dependency projection이다.
>
> `src/utils/queryContext.ts:44–87`
>
> **보존 조건.** 순서가 바뀌면 instruction scope와 cache prefix가 달라질 수 있다.

> **A09 · API message normalization**
>
> 내부 history를 provider가 수용하는 role/content pairing으로 낮춘다.
>
> `src/utils/messages.ts:1989–2028`
>
> **보존 조건.** 각 tool_result는 대응 tool_use 뒤에 온다.

> **A14 · Message normalization lookup 구축**
>
> 반복적인 sibling/tool 조회를 index로 바꿔 대화 history를 선형에 가깝게 처리한다.
>
> `src/utils/messages.ts:1170–1215`
>
> **보존 조건.** index는 정확히 같은 immutable message snapshot에 대응한다.

장

query state machine과 typed stream 재구성

반복되는 model/tool phase와 조각난 provider event는 각각 명시적 state와 variant 검증으로 불완전한 상태의 전파를 제한한다.

목차

while loop를 위에서 아래로 읽으면 여러 continue가 같은 의미처럼 보인다. 실제로는 retry, compaction, tool result, stop hook가 서로 다른 다음 state를 요구한다. queryLoop의 State object는 messages와 turnCount만이 아니라 pending summary와 recovery counter, transition reason을 한 결정 단위로 묶는다.

stream 쪽에서는 완성 message가 한 번에 오지 않는다. index별 builder는 start/delta/stop 순서를 검증하고, tool input JSON을 완료 전까지 실행 값으로 취급하지 않는다. UI에 보이는 block 완료와 usage·stopReason까지 포함한 terminal 완료가 다른 시점이라는 사실이 핵심이다.

그림 3 · 한 turn의 종단간 전이에서 이 장의 정적 책임과 동적 연결을 먼저 확인한다.

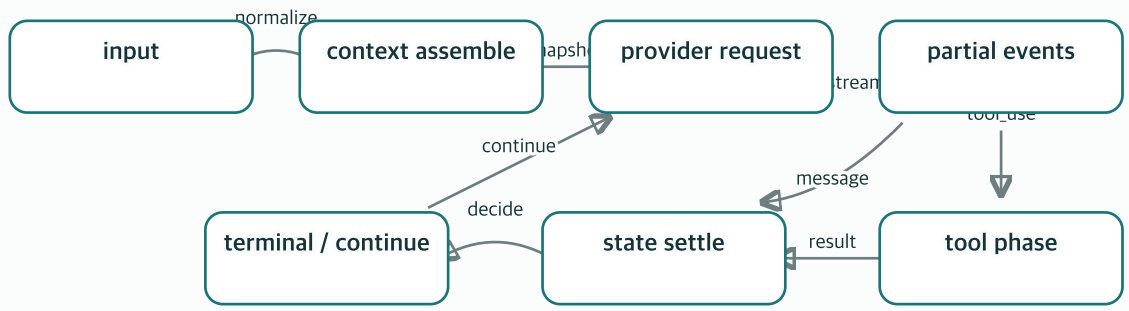


그림 3
한 turn의 종단간 전이 — 사용자 입력 뒤에는 하나의 직선이 아니라 model·tool·compact·retry·terminal 사이의 상태 전이가 있다. 각 화살표는 다음 phase가 믿을 수 있는 state를 완성해야 한다.

이 장의 결정점

A10 · queryLoop 상태 전이

무한 while보다 중요한 것은 continue마다 새 State를 만드는 전이 규칙이다.

`src/query.ts:252–321`

보존 조건. tool_use/tool_result pairing 없이 다음 provider call로 가지 않는다.

A11 · tool-use/tool-result pairing

외부 effect는 둘 사이에서 일어나지만 대화 protocol에는 같은 id로 닫힌 원자가 필요하다.

`src/services/tools/toolExecution.ts:337–410`

보존 조건. unknown tool도 silent drop하지 않는다.

A39 · 부분 content block의 typed reconstruction

provider event sequence를 index별 mutable builder에 누적하고 block completion에서 immutable-looking AssistantMessage로 투영한다.

`src/services/api/claude.ts:1940–2054`

보존 조건. delta index에는 먼저 시작된 block이 존재한다.

> **A12 · API retry와 backoff**
>
> retry 여부는 status code뿐 아니라 source, fast mode, retry-after, consumer-visible event에 좌우된다.
>
> `src/services/api/withRetry.ts:170–220`
>
> **보존 조건.** abort는 새 retry를 시작하지 않는다.

언어 메커니즘

yield는 여러 번 반환하는 문법이 아니다

async generator를 이해하려면 함수 호출이 끝났는지, 실행 frame이 멈춘 채 보존되었는지, 소비자가 어떤 iterator result를 받았는지를 분리해야 한다.

먼저 return과 suspension을 구분한다

일반 함수의 return은 현재 호출 frame을 종료하고 호출자에게 제어를 돌려준다. 반면 generator의 yield는 현재 instruction pointer, local variable, try/finally 문맥을 보존한 suspension point다. 소비자가 다음 next()를 호출할 때 같은 frame이 yield 다음 지점에서 다시 진행된다.

따라서 '함수를 여러 번 나눠 실행한다'는 직관은 입문용으로는 도움이 되지만 충분하지 않다. 중요한 것은 여러 번의 새 함수 호출이 아니라 하나의 실행 frame이 pause와 resume를 반복한다는 점이다. 이 차이가 cleanup, error injection, UI의 partial projection을 설명한다.

사례 1 — next()는 보존된 local state를 다시 연다

사례 1

```
function* count() {
  let n = 1
  yield n
  n += 1
  return n
}

const it = count()
```

```
it.next() // { value: 1, done: false }
it.next() // { value: 2, done: true }
```

첫 next() 뒤 n과 instruction pointer는 사라지지 않는다. 두 번째 next()는 새 count() 호출이 아니라 같은 frame을 resume하고, return이 terminal iterator result를 만든다. 그 뒤의 next()는 이미 닫힌 generator에서 더 진행할 코드를 찾지 않는다.

사례 2 — return은 terminal reason을 전달한다

사례 2

```
function* task() {
  yield { type: 'progress', value: 50 }
  return { type: 'terminal', reason: 'complete' }
}

const it = task()
it.next() // progress, done: false
it.next() // terminal, done: true
```

yielded value는 관찰 가능한 중간 event이고 return value는 같은 반복의 terminal 결과다. for…of와 for await…of는 보통 done:true의 value를 loop body에 전달하지 않으므로 terminal reason이 필요하다면 수동 next() 또는 별도 wrapper contract가 필요하다.

사례 3 — throw는 소비자와 frame 사이의 오류 경로를 연다

사례 3

```
function* guarded() {
  try {
    yield 'waiting'
  } catch (error) {
    yield `recovered: ${error.message}`
  } finally {
    cleanup()
  }
}

const it = guarded()
it.next()
it.throw(new Error('cancel'))
```

iterator.throw(error)는 멈춘 yield 지점에서 예외가 발생한 것처럼 frame 안으로 오류를 주입한다. 내부 catch가 받지 않으면 소비자의 next/throw 호출이 reject·throw되고 generator는 닫힌다. finally는 정상 return, 외부 return(), uncaught throw에서

cleanup 경계가 된다.

사례 4 — async generator는 stream event와 terminal을 다른 채널로 둔다

사례 4

```
async function* query() {
  yield { type: 'message_delta', text: '안' }
  yield { type: 'message_delta', text: '녕' }
  return { reason: 'end_turn' }
}

for await (const event of query()) {
  projectToUI(event)
}
```

각 yield는 provider에서 온 partial state를 UI나 observer가 즉시 projection할 기회를 준다. 하지만 두 글자가 보였다는 사실은 turn이 terminal이라는 뜻이 아니다. return 또는 surrounding protocol이 end_turn을 확정하고, throw는 error UI와 cleanup·retry 경로를 연다.

속성	yield value	return terminal	throw error
함수 상태	frame 보존·suspend	정상 종료	catch되지 않으면 abrupt 종료
소비 결과	{ value, done:false }	{ value, done:true }	next/throw의 예외·reject
query 의미	stream message·progress	terminal reason	실패·중단 전파
UI 영향	즉시 projection 가능	turn 마감	error UI·cleanup·retry 판단

> **프로젝트로 돌아가기.** 실제 query와 provider stream에서는 yield 수를 세는 대신 각 event가 어떤 partial state를 노출하고, canonical message가 언제 닫히며, return/throw가 어느 owner에게 terminal을 전달하는지 추적해야 한다. A10의 turn State와 A39의 typed accumulator가 이 문법 메커니즘을 프로젝트 수준의 보장으로 바꾼다.

queryLoop 상태 전이

한 user turn 안에서 model 요청, tool 실행, retry, compaction, stop hook가 반복된다. 함수의 while loop만 따라가면 같은 continue가 서로 다른 상태 의미를 갖는다는 사실을 놓치기 쉽다.

> **독자 질문.** 명시적 State object는 여러 continue 경로에서 어떤 전이 누락을 막고, terminal reason은 어디에서 확정되는가?

이 장의 시작 · A10 ATLAS · 목차

결정적인 코드를 읽기 전에

상태기계와 transition invariant

같은 while iteration이라도 현재 phase와 state에 따라 허용되는 다음 행동이 다르다. 전이는 일부 field만 바꾸는 assignment가 아니라 다음 phase가 믿을 수 있는 상태 전체를 만드는 일이다.

> **source에서 볼 지점.** 252-321행의 초기 State가 이후 모든 continue branch에서 완전하게 갱신되는지 추적한다.

event sourcing과 현재 state의 차이

yield된 event 목록은 관찰 history이고 State는 다음 결정을 위한 현재 snapshot이다. 둘은 서로를 보완하지만 자동으로 같지 않다.

> **source에서 볼 지점.** partial event를 냈지만 canonical messages가 아직 갱신되지 않은 구간을 구분한다.

같은 continue라도 누락된 field 하나가 다음 phase의 의미를 바꾼다

전이 함수 없이 object를 부분 갱신할 때 pending 상태가 남는 경우를 축소해 본다.

A10 축소 예제

```
let s = { phase: 'model', pendingTool: null, reason: null }
// model이 tool call을 냄
```

```
s = { ...s, phase: 'tool', pendingTool: 't1' }
// retry branch가 phase만 되돌림
s = { ...s, phase: 'model' }
// pendingTool === 't1'이 남아 다음 결과와 중복 pairing될 수 있음
```

지점	관찰	해석
초기	model phase, pending 없음	새 provider 응답을 받을 수 있다.
tool 전이	pendingTool=t1	tool result가 t1을 닫아야 한다.
불완전한 retry	phase만 model로 돌아감	과거 pending identity가 새 attempt에 남는다.

> **프로젝트로 돌아가기.** 실제 State의 각 continue site에서 transition reason과 pending/compact field가 함께 갱신되는지 확인하는 이유다.

고정 source에서 결정점을 찾는다

여러 model request와 tool phase를 한 user turn 안에서 반복하면서 recovery·compact·budget 상태를 보존한다.

src/query.ts:252-321 · queryLoop state initialization

```
252    // Immutable params — never reassigned during the query loop.
253    const {
254      systemPrompt,
255      userContext,
256      systemContext,
257      canUseTool,
258      fallbackModel,
259      querySource,
260      maxTurns,
261      skipCacheWrite,
262    } = params
263    const deps = params.deps ?? productionDeps()
264
265    // Mutable cross-iteration state. The loop body destructures ↩
this at the top
266    // of each iteration so reads stay bare-name (`messages`, ↩
`toolUseContext`).
267    // Continue sites write `state = { ... }` instead of 9 ↩
separate assignments.
268    let state: State = {
269      messages: params.messages,
270      toolUseContext: params.toolUseContext,
271      maxOutputTokensOverride: params.maxOutputTokensOverride,
272      autoCompactTracking: undefined,
273      stopHookActive: undefined,
274      maxOutputTokensRecoveryCount: 0,
275      hasAttemptedReactiveCompact: false,
276      turnCount: 1,
277      pendingToolUseSummary: undefined,
```

```
278       transition: undefined,
279     }
280     const budgetTracker = feature('TOKEN_BUDGET') ? ↩
createBudgetTracker() : null
281
      … 40개 행은 source link에서 이어 읽기
```

고정 revision의 전체 범위 열기

구조 안에서 누가 소유하는가

OBSERVED	queryLoop가 messages, turnCount, compaction/recovery 추적, pending summary, transition reason을 한 State로 소유한다.
INFERENCE	REPL이나 headless shell이 turn을 시작하지만, turn 내부의 model/tool/settle 상태기계는 queryLoop가 소유한다.

입력. messages · system/user/system context · ToolUseContext · limits

유지 state. messages · turnCount · compact tracking · max-output recovery count · pending summary · transition reason

상태가 어떻게 바뀌는가

1. immutable config를 snapshot한다.
2. iteration state를 destructure한다.
3. provider stream을 시작한다.
4. assistant blocks와 tool calls를 수집한다.
5. 필요하면 tools를 실행하고 message/state를 갱신한다.
6. stop·compact·retry 조건에 따라 terminal 또는 다음 iteration으로 전이한다.

명시적 State object로 여러 continue site의 partial assignment 위험을 줄인다.

보존 조건과 깨지는 방식

- tool_use/tool_result pairing 없이 다음 provider call로 가지 않는다.
- 각 continue는 transition reason과 새 state를 남긴다.
- terminal path는 더 이상 effect를 시작하지 않는다.

반증 가능한 실패

- state field를 continue site 하나에서 누락
- partial message 뒤 retry로 중복 관찰
- stop hook가 terminal을 무한 지연

배경 이론은 source로 돌아오기 위한 도구다

- **state machine** — 현재 상태와 입력이 허용된 다음 상태와 action을 결정하는 모델이다.
- **event sourcing** — 현재 값만 저장하지 않고 변화 event로부터 projection을 재구성하는 접근이다.
- **transition invariant** — 한 phase에서 다음 phase로 넘어갈 때 함께 성립해야 하는 조건이다.

전이 enum과 reducer를 추출하고 model/tool/settle phase를 property test한다.

실행과 운영에서 생기는 결과

OBSERVED	model event를 소비하고 tool phase를 실행하며 budget·compact·stop 조건에 따라 다음 request 또는 terminal로 전이한다.
INFERENCE	tool effect는 turn state와 분리된 외부 세계에 남을 수 있어 retry 전에 commit 여부를 알아야 한다.
UNKNOWN	partial message 뒤 retry, stop hook의 반복 지연, continue site의 field 누락은 중복 관찰이나 무한 turn을 만든다.
RECOMMENDATION	phase, transition reason, attempt id, pending tool ids, terminal reason을 한 trace로 묶어야 한다.

지금 재구성할 수 있는 시스템 상태

OBSERVED	명시적 State 초기화와 다수의 model/tool/compact 전이가 존재한다.
INFERENCE	모든 continue site가 필요한 field를 갱신할 때 다음 phase가 일관된 snapshot을 본다.
UNKNOWN	feature-gated 모든 전이와 production에서의 rare loop 조합은 이 발췌만으로 닫을 수 없다.
RECOMMENDATION	reducer 추출 전 현재 continue site를 전이 표로 만들고 property test 입력으로 고정해야 한다.

다음 코드를 읽을 때 가져갈 질문

- partial assistant message 뒤 provider retry가 시작되면 새 attempt와 기존 event를 어떤 identity로 구분할 것인가?
- stop hook가 다시 query를 요구할 때 turnCount와 terminal reason은 어느 전이에서 바뀌어야 하는가?

심층 코드 독해 · A39

부분 content block의 typed reconstruction

provider는 완성 message가 아니라 message_start, 여러 index의 block start/delta/stop, 늦은 message_delta를 보낸다. UI에는 partial content를 즉시 보여 주면서 transcript에는 protocol-valid한 canonical message를 남겨야 한다.

> **독자 질문.** index별 mutable accumulator와 object identity가 partial visibility, tool JSON 완성, late metadata 정산을 어떻게 동시에 보존하는가?

이 장의 시작 · A39 ATLAS · 목차

결정적인 코드를 읽기 전에

discriminated union과 runtime validation

type tag는 각 delta가 어떤 accumulator variant와 결합할 수 있는지 code branch를 좁힌다. 하지만 network가 start-before-delta 순서를 지킨다는 사실은 TypeScript가 보장하지 않으므로 runtime check가 필요하다.

> **source에서 볼 지점.** 1940-2054행에서 index lookup, type 비교, missing block error가 함께 나타나는 지점을 본다.

mutable builder와 referential identity

stream 중에는 조각을 한 object에 누적하고 늦은 metadata를 같은 reference에 반영한다. immutable replacement는 안전해 보이지만 이미 그 reference를 잡은 지연 serializer와 identity를 갈라놓을 수 있다.

> **source에서 볼 지점.** block stop 이후 yield와 message_delta mutation, transcript queue serialization timing을 함께 추적한다.

보이는 완료와 metadata의 terminal 완료는 같은 시점이 아니다

text block stop 뒤 usage가 늦게 오는 event sequence를 축소한다.

A39 축소 예제

```
start(message)
startBlock(0, 'text')
delta(0, '안녕')
stopBlock(0)       // content를 yield
messageDelta({ usage: 12, stop: 'end_turn' }) // 같은 message identity 갱신
```

지점	관찰	해석
block start	index 0 builder 생성	delta를 받을 valid state가 열린다.
block stop	text content가 화면에 보임	message 전체 metadata는 아직 terminal이 아니다.
message delta	usage/stopReason 갱신	지연 transcript serializer가 같은 identity를 봐야 한다.

> **프로젝트로 돌아가기.** yield 직후 deep copy나 object replacement가 late metadata를 잃게 할 수 있는 이유를 실제 queue와 함께 읽는다.

고정 source에서 결정점을 찾는다

text, thinking, tool input JSON, connector block이 서로 다른 delta로 도착하는 동안 type과 index를 잃지 않고 내부 message를 만든다.

src/services/api/claude.ts:1940-2054 · message start와 content block 초기화

```
1940        for await (const part of stream) {
1941          resetStreamIdleTimer()
1942          const now = Date.now()
1943
1944          // Detect and log streaming stalls (only after first ↩
 event to avoid counting TTFB)
1945          if (lastEventTime !== null) {
1946            const timeSinceLastEvent = now - lastEventTime
1947            if (timeSinceLastEvent > STALL_THRESHOLD_MS) {
1948              stallCount++
1949              totalStallTime += timeSinceLastEvent
1950              logForDebugging(
1951                `Streaming stall detected: ${(timeSinceLastEvent ↩
 / 1000).toFixed(1)}s gap between events (stall #${stallCount})`,
1952                { level: 'warn' },
1953              )
1954              logEvent('tengu_streaming_stall', {
1955                stall_duration_ms: timeSinceLastEvent,
1956                stall_count: stallCount,
1957                total_stall_time_ms: totalStallTime,
1958                event_type:
1959                  part.type as AnalyticsMetadata_I_VERIFIED_THIS_ ↩
 IS_NOT_CODE_OR_FILEPATHS,
1960                model:
1961                  options.model as ↩
 AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
1962                request_id: (streamRequestId ??
1963                  'unknown') as AnalyticsMetadata_I_VERIFIED_THIS ↩
 _IS_NOT_CODE_OR_FILEPATHS,
1964              })
1965            }
1966          }
1967          lastEventTime = now
1968
1969          if (isFirstChunk) {
      … 85개 행은 source link에서 이어 읽기
```

고정 revision의 전체 범위 열기

구조 안에서 누가 소유하는가

OBSERVED	claude.ts의 stream consumer가 partialMessage, contentBlocks, usage, stopReason, newMessages와 variant별 상태를 소유한다.
INFERENCE	provider SDK는 event를 생산하지만 local consumer가 canonical block builder를 소유하고 UI/transcript는 그 projection을 소비한다.

입력. message_start · content_block_start/delta/stop · message_delta · tool registry와 request id

유지 state. partialMessage · index별 contentBlocks · usage · stopReason · newMessages · research/advisor state

상태가 어떻게 바뀌는가

1. message_start에서 envelope와 초기 usage를 잡는다.
2. content_block_start에서 variant별 빈 accumulator를 만든다.
3. delta discriminant와 기존 block type이 맞는지 검사한 뒤 text·JSON·thinking을 누적한다.
4. content_block_stop에서 해당 block을 normalize해 AssistantMessage를 만들고 yield한다.
5. message_delta에서 최종 usage와 stop reason을 이미 만든 마지막 message에 반영한다.
6. 원본 provider event도 stream_event로 별도 yield한다.

provider SDK의 mutable/중복 동작을 그대로 퍼뜨리지 않고 local accumulator와 explicit mismatch error로 정규화한다.

보존 조건과 깨지는 방식

- delta index에는 먼저 시작된 block이 존재한다.
- delta type과 accumulator variant가 일치한다.
- tool input JSON은 block 완료 전까지 문자열 조각이며 실행 input으로 사용되지 않는다.
- 최종 usage와 stop reason이 transcript가 직렬화할 같은 object reference에 반영된다.

반증 가능한 실패

- start 없는 delta
- index 재사용

- partial JSON을 조기 parse
- message_delta 이전에 object reference를 복제해 최종 metadata 유실

배경 이론은 source로 돌아오기 위한 도구다

- **state machine** — 현재 상태와 입력이 허용된 다음 상태와 action을 결정하는 모델이다.
- **tagged union** — discriminant가 가능한 payload variant와 처리 branch를 결합하는 자료형이다.
- **referential integrity** — identity와 참조 관계가 늦은 갱신·직렬화 뒤에도 일관되게 유지되는 성질이다.

block builder를 variant별 reducer로 분리하고 event sequence property test로 start→delta*→stop 규약과 malformed 순서를 검증한다.

실행과 운영에서 생기는 결과

OBSERVED	event마다 block map을 갱신하고 완료 block/message를 yield하며 idle watchdog를 reset한다.
INFERENCE	UI projection, transcript queue, telemetry timer가 서로 다른 시점에 같은 message를 관찰한다.
UNKNOWN	start 없는 delta, index 재사용, partial JSON 조기 parse, stale reference, watchdog cleanup 누락은 서로 다른 불완전 상태를 만든다.
RECOMMENDATION	request id, event index/type, builder state, yield identity, terminal metadata, timeout reason을 연결해야 한다.

지금 재구성할 수 있는 시스템 상태

OBSERVED	typed block accumulator, missing-block runtime error, completion yield, late metadata mutation, raw event projection, idle watchdog가 존재한다.
INFERENCE	provider sequence와 local variant checks가 일치하고 serializer가 같은 object identity를 볼 때 canonical message가 완성된다.
UNKNOWN	malformed sequence의 production 빈도, feature-gated connector/advisor 활성화, 모든 runtime의 queue timing은 확인되지 않는다.
RECOMMENDATION	variant별 reducer와 event sequence property test를 도입하기 전 1940-2302행과 transcript serializer timing을 한 trace로 재검증해야 한다.

다음 코드를 읽을 때 가져갈 질문

- stopBlock 전에 partial tool JSON을 parse하면 어떤 invalid input이 실행 가능한 값처럼 보이는가?
- message object를 immutable replacement하면 이미 queue에 들어간 reference의 usage와 stopReason은 어떻게 되는가?

III

도구, 동시성, 권한, 파일 변경은 어떻게 effect를 제한하는가

실행 가능성, 허가, sandbox 실행, commit, 관찰 결과를 서로 다른 경계로 읽는다.

목차로

장

도구 선택과 동시 실행의 보존 조건

독립 tool을 병렬화해도 입력 순서, identity, 실패 의미를 보존하지 못하면 latency 최적화가 semantic regression이 된다.

목차

도구 호출 목록을 곧바로 Promise.all에 넣는 것만으로 concurrency가 설계되지는 않는다. 병렬 실행 가능한 집합을 분리하고, 결과를 원 입력 identity에 다시 결합하며, 일부 실패가 전체 turn을 어떻게 바꾸는지 명시해야 한다.

A17의 sequence-concurrency merge는 결과가 완료된 순서와 독자에게 제시될 순서를 분리한다. A18의 권한 경계는 실행 전에 effect class와 policy를 결합한다. 두 알고리즘은 속도와 authority를 각각 맡지만 tool result identity에서 만난다.

이 장의 결정점

> **A15 · Generic factory의 default 합성**
>
> buildTool은 runtime object spread와 type-level spread를 맞춘다.
>
> `src/Tool.ts:721–791`
>
> **보존 조건.** def가 default를 덮어쓴다.

> **A16 · 입력별 concurrency batch partition**
>
> 병렬성은 tool 이름이 아니라 parse된 구체 input의 commutativity를 근사한다.
>
> `src/services/tools/toolOrchestration.ts:84–116`
>
> **보존 조건.** parse 실패는 unsafe다.

> **A17 · 병렬 결과의 deterministic context merge**
>
> 병렬 call 중 context mutation을 즉시 적용하지 않고 tool order대로 지연 합성한다.
>
> `src/services/tools/toolOrchestration.ts:19–81`
>
> **보존 조건.** 동시에 currentContext를 write하지 않는다.

> **A18 · 공통 permission-and-call pipeline**
>
> schema가 맞는 입력을 곧바로 실행하지 않고 hook·validation·authority·call 순서를 통과시킨다.
>
> `src/services/tools/toolExecution.ts:599–720`
>
> **보존 조건.** 사용자가 승인한 input과 call input의 identity를 추적한다.

심층 코드 독해 · A17

병렬 결과의 deterministic context merge

병렬 tool은 progress와 result를 완료되는 즉시 보여 주고 싶지만 shared ToolUseContext를 완료 순서대로 합치면 실행마다 최종 state가 달라질 수 있다.

> **독자 질문.** 낮은 latency의 event 순서와 재현 가능한 canonical merge 순서를 어떻게 분리하는가?

이 장의 시작 · A17 ATLAS · 목차

결정적인 코드를 읽기 전에

two-phase merge와 deterministic reduction

각 worker는 결과와 modifier를 독립적으로 계산하고, shared state 반영은 고정된 순서로 한 번 더 reduce한다. completion order를 commit order로 쓰지 않는 것이 핵심이다.

> **source에서 볼 지점.** 19-81행에서 Promise completion과 modifier application이 서로 다른 순서를 쓰는지 본다.

완료 순서와 merge 순서를 분리하면 같은 최종 context를 얻는다

두 tool이 서로 다른 시간에 끝나지만 선언 순서로 modifier를 적용하는 축소 예제다.

A17 축소 예제

```
results = await Promise.all([run('A', 30), run('B', 5)])
// B progress가 먼저 보여도
for (const result of results) context = result.modify(context)
// merge는 입력 배열 A → B 순서
```

지점	관찰	해석
5ms	B가 먼저 완료	progress latency는 줄지만 아직 canonical context order를 뜻하지 않는다.
30ms	A가 완료	모든 modifier가 준비된다.
merge	A 다음 B로 reduce	반복 실행에서도 같은 context order를 얻는다.

> **프로젝트로 돌아가기.** modifier queue가 toolUseID와 입력 순서를 어떻게 보존하는지 실제 구현에서 확인한다.

고정 source에서 결정점을 찾는다

progress/result는 병렬로 내보내면서 shared ToolUseContext의 갱신 순서를 안정화한다.

src/services/tools/toolOrchestration.ts:19-81 · runTools batch execution

```
 19  export async function* runTools(
 20    toolUseMessages: ToolUseBlock[],
 21    assistantMessages: AssistantMessage[],
 22    canUseTool: CanUseToolFn,
 23    toolUseContext: ToolUseContext,
 24  ): AsyncGenerator<MessageUpdate, void> {
 25    let currentContext = toolUseContext
 26    for (const { isConcurrencySafe, blocks } of partitionToolCalls(
 27      toolUseMessages,
 28      currentContext,
 29    )) {
 30      if (isConcurrencySafe) {
 31        const queuedContextModifiers: Record<
 32          string,
 33          ((context: ToolUseContext) => ToolUseContext)[]
 34        > = {}
 35        // Run read-only batch concurrently
 36        for await (const update of runToolsConcurrently(
 37          blocks,
 38          assistantMessages,
 39          canUseTool,
 40          currentContext,
 41        )) {
 42          if (update.contextModifier) {
 43            const { toolUseID, modifyContext } = update ↩
.contextModifier
 44            if (!queuedContextModifiers[toolUseID]) {
 45              queuedContextModifiers[toolUseID] = []
 46            }
 47            queuedContextModifiers[toolUseID].push(modifyContext)
 48          }
    … 33개 행은 source link에서 이어 읽기
```

고정 revision의 전체 범위 열기

구조 안에서 누가 소유하는가

OBSERVED	runTools가 currentContext와 toolUseID별 modifier queue를 소유하며 batch result를 deterministic order로 합친다.
INFERENCE	tool execution은 병렬이지만 context commit은 orchestrator의 단일 merge 경계다.

입력. concurrency-safe batch · MessageUpdateLazy

유지 state. currentContext · toolUseID별 modifier queue

상태가 어떻게 바뀌는가

1. 각 tool을 bounded concurrent로 실행한다.
2. message는 즉시 yield한다.
3. context modifier는 toolUseID별로 모은다.
4. batch가 끝나면 원래 block 순서로 modifier를 적용한다.

관찰 event의 latency와 canonical context의 determinism을 분리한다.

보존 조건과 깨지는 방식

- 동시에 currentContext를 write하지 않는다.
- merge order는 completion timing이 아니라 input order다.
- unsafe tool은 이 경로를 사용하지 않는다.

반증 가능한 실패

- modifier가 실제로 non-commutative
- tool이 context 밖 global을 직접 변경
- progress order를 result order로 오해

배경 이론은 source로 돌아오기 위한 도구다

- **deterministic reduction** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.
- **two-phase merge** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.
- **bounded concurrency** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.

modifier가 읽고 쓰는 context slice를 선언하고 conflict를 검증한다.

실행과 운영에서 생기는 결과

OBSERVED	tool은 병렬 실행되고 lazy update가 도착하지만 context modifier는 안정된 batch 순서로 적용된다.
INFERENCE	tool이 context 밖 global을 직접 쓰면 two-phase 경계를 우회한다.
UNKNOWN	non-commutative modifier나 progress order를 result order로 오해하면 UI와 canonical state가 갈라진다.

RECOMMENDATION	started/completed/projected/merged 시각과 order를 분리해 기록해야 한다.

지금 재구성할 수 있는 시스템 상태

OBSERVED	병렬 event projection과 toolUseID별 modifier queue가 분리돼 있다.
INFERENCE	modifier가 선언된 context만 변경하고 merge 순서가 안정적일 때 최종 state가 결정적이다.
UNKNOWN	구체 tool의 숨은 global write와 modifier 간 실제 commutativity는 모두 확인되지 않는다.
RECOMMENDATION	modifier마다 read/write context slice를 선언하고 충돌 corpus를 만들어야 한다.

다음 코드를 읽을 때 가져갈 질문

- B progress가 먼저 보인 뒤 A가 실패하면 UI와 canonical context는 각각 무엇을 남겨야 하는가?
- 두 modifier가 같은 field를 쓴다면 fixed order는 determinism만 주는가, 올바른 의미도 보장하는가?

심층 코드 독해 · A18

공통 permission-and-call pipeline

모델이 만든 JSON은 아직 실행 권한이 없는 외부 입력이다. schema parsing, hook 변형, permission, 실행이 다른 순서로 배치되면 승인한 값과 실행한 값이 달라질 수 있다.

> **독자 질문.** unknown input이 승인된 executable input으로 바뀌는 각 단계에서

> representation과 authority는 어떻게 변하는가?

이 장의 시작 · A18 ATLAS · 목차

결정적인 코드를 읽기 전에

reference monitor와 TOCTOU

모든 effect 요청은 우회할 수 없는 공통 검사점을 지나야 한다. 검사한 값과 사용하는 값 사이에서 mutation이 일어나면 time-of-check와 time-of-use가 갈라진다.

> **source에서 볼 지점.** 599-720행에서 parse·hook·permission·call 사이에 input identity가 언제 바뀌는지 본다.

승인 뒤 input이 바뀌면 같은 tool call도 다른 effect가 된다

검사한 path와 실행한 path가 달라지는 최소 반례다.

A18 축소 예제

```
let input = { path: '/safe/a' }
permit(input)
input.path = '/secret/b'
callTool(input) // 허용 판단과 실행 대상이 다름
```

지점	관찰	해석
parse	/safe/a를 구조화	아직 authority 없음
permission	/safe/a를 허용	결정은 이 representation에만 유효
mutation	/secret/b로 변경	기존 decision을 재사용하면 TOCTOU 발생

> **프로젝트로 돌아가기.** 실제 pipeline에서 hook updatedInput이 permission 전인지 후인지와 digest가 유지되는지 추적한다.

고정 source에서 결정점을 찾는다

모델이 만든 unknown JSON을 현재 workspace에서 실행 가능한 승인된 input으로 바꾼다.

src/services/tools/toolExecution.ts:599-720 · checkPermissionsAndCallTool 앞부분

```
599  async function checkPermissionsAndCallTool(
600    tool: Tool,
601    toolUseID: string,
602    input: { [key: string]: boolean | string | number },
603    toolUseContext: ToolUseContext,
604    canUseTool: CanUseToolFn,
605    assistantMessage: AssistantMessage,
606    messageId: string,
607    requestId: string | undefined,
608    mcpServerType: McpServerType,
609    mcpServerBaseUrl: ReturnType<typeof getLoggingSafeMcpBaseUrl>,
610    onToolProgress: (
611      progress: ToolProgress<ToolProgressData> | ↩
ProgressMessage<HookProgress>,
612    ) => void,
613  ): Promise<MessageUpdateLazy[]> {
614    // Validate input types with zod (surprisingly, the model is ↩
not great at generating valid input)
615    const parsedInput = tool.inputSchema.safeParse(input)
616    if (!parsedInput.success) {
617      let errorContent = formatZodValidationError(tool.name, ↩
parsedInput.error)
618
619      const schemaHint = buildSchemaNotSentHint(
620        tool,
621        toolUseContext.messages,
622        toolUseContext.options.tools,
623      )
624      if (schemaHint) {
625        logEvent('tengu_deferred_tool_schema_not_sent', {
626          toolName: sanitizeToolNameForAnalytics(tool.name),
627          isMcp: tool.isMcp ?? false,
628        })
     … 92개 행은 source link에서 이어 읽기
```

고정 revision의 전체 범위 열기

구조 안에서 누가 소유하는가

OBSERVED	checkPermissionsAndCallTool이 parsed/updated input, permission result, progress, duration을 순서대로 소유한다.
INFERENCE	tool 구현은 실제 effect를 수행하고 공통 pipeline은 validation과 authority ordering을 소유한다.

입력. Tool · toolUseID · raw input · ToolUseContext · canUseTool

유지 state. parsed/updated input · permission result · progress · duration

상태가 어떻게 바뀌는가

1. runtime schema와 validateInput을 적용한다.
2. pre-tool hook가 입력/결정을 수정할 수 있다.
3. tool-specific permission을 계산한다.
4. 필요하면 human/host canUseTool을 호출한다.
5. 승인된 최종 input으로 call한다.
6. success/failure hook와 result projection을 수행한다.

도구 구현마다 반복하지 않고 ordering-sensitive control을 중앙화한다.

보존 조건과 깨지는 방식

- 사용자가 승인한 input과 call input의 identity를 추적한다.
- deny/abort에서도 terminal tool_result를 만든다.
- validation과 authority를 같은 Boolean으로 축약하지 않는다.

반증 가능한 실패

- 승인 뒤 input mutation
- hook timeout 정책 불명확
- validation과 permission 사이 TOCTOU

배경 이론은 source로 돌아오기 위한 도구다

- **reference monitor** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.
- **pipeline** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.
- **policy enforcement point** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.

각 transformation에 input digest와 provenance를 기록하는 decision trace를 만든다.

실행과 운영에서 생기는 결과

OBSERVED	raw input을 parse/normalize하고 hook와 permission을 거쳐 tool을 호출하며 progress/result를 투영한다.
INFERENCE	callTool 진입이 외부 effect 경계이며 그 직전 input과 decision이 같은 identity여야 한다.

UNKNOWN	승인 후 mutation, timeout policy 모호성, executor 우회는 reference monitor를 깨뜨린다.
RECOMMENDATION	각 transformation의 before/after digest와 decision source를 남겨야 한다.

지금 재구성할 수 있는 시스템 상태

OBSERVED	공통 permission-and-call pipeline과 ordering-sensitive 단계가 있다.
INFERENCE	모든 tool executor가 이 경계를 통과하고 최종 input에 permission이 적용될 때 authority가 맞는다.
UNKNOWN	별도 executor와 feature variant의 우회 여부는 선택 범위만으로 닫히지 않는다.
RECOMMENDATION	새 transformation 추가 시 permission 재평가 조건과 provenance trace를 먼저 정의해야 한다.

다음 코드를 읽을 때 가져갈 질문

- hook가 path를 바꾼다면 기존 permission result를 재사용할 수 있는가?
- schema parsing이 default 값을 추가하면 사용자가 승인 화면에서 그 값을 볼 수 있는가?

장

permission, hook, sandbox는 하나의 yes/no가 아니다

요청의 형태, 정책, 사용자 승인, hook 변환, sandbox 제한은 서로 다른 authority와 실패를 가진다.

목차

permission 결과가 allow라고 해서 effect가 이미 일어난 것은 아니다. pre-hook는 입력을 거부하거나 바꿀 수 있고, sandbox는 허용된 호출의 실제 능력을 제한하며, executor 실패는 정책 판단과 무관하게 발생한다. 이 차이를 한 boolean으로 접으면 retry와 감사 로그가 거짓이 된다.

결정 표는 순서도의 장식이 아니라 반례를 만드는 도구다. 같은 tool name이라도 resource, path, command prefix, session mode에 따라 다른 branch를 타며, 각 branch는 어떤 근거로 허용·거부되었는지를 남겨야 한다.

그림 4 · 권한 판단과 effect commit에서 이 장의 정적 책임과 동적 연결을 먼저 확인한다.

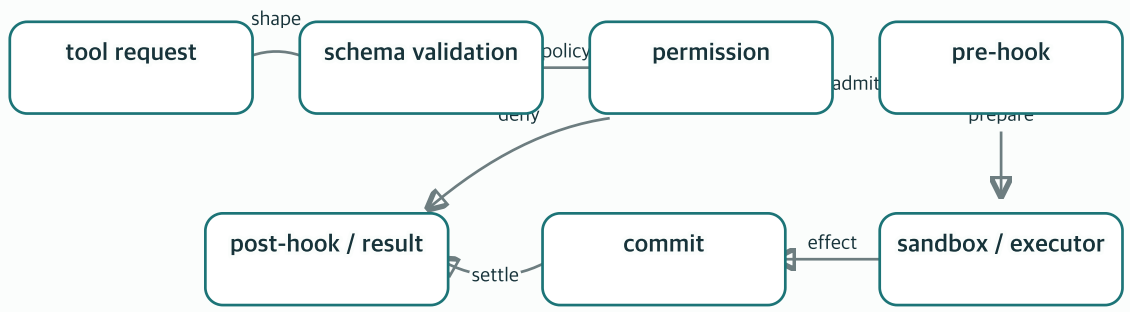


그림 4
권한 판단과 effect commit — 도구가 실행 가능하다는 사실과 외부 세계에 결과가 정산되었다는 사실은 다르다. validation·permission·hook·sandbox·commit이 서로 다른 실패와 재시도 경계를 만든다.

이 장의 결정점

A19 · Rule source aggregation과 behavior precedence

설정 source 순서와 allow/deny/ask 평가 순서는 같은 개념이 아니다.

`src/utils/permissions/permissions.ts:109–131`

보존 조건. rule에는 source가 남는다.

A20 · Tool-level deny/ask short circuit

도구 전체 rule은 구체 input permission보다 앞서 빠른 terminal decision을 만든다.

`src/utils/permissions/permissions.ts:1070–1120`

보존 조건. deny는 하위 allow가 조용히 덮지 않는다.

A21 · Compound command decision 합성

복합 shell 명령은 subcommand별 결과를 계산하고 전체 decision으로 접는다.

`src/tools/BashTool/bashPermissions.ts:2430–2485`

보존 조건. 부분 allow로 compound command를 실행하지 않는다.

> **A42 · Hook fan-out과 permission decision 합성**
>
> 일치하는 여러 hook을 병렬 실행하되 각 결과를 progress·context·input mutation·permission lattice로 순차 투영한다.
>
> `src/utils/hooks.ts:1952–2075`
>
> **보존 조건.** 신뢰되지 않은 workspace의 hook은 실행하지 않는다.

> **A43 · 설정과 policy를 sandbox runtime capability로 투영**
>
> permission rule과 sandbox setting의 서로 다른 path 문법을 해석하고 network·read·write capability를 runtime config로 낮춘다.
>
> `src/utils/sandbox/sandbox-adapter.ts:172–260`
>
> **보존 조건.** managed-only가 켜지면 낮은 authority source가 allow 범위를 넓히지 못한다.

장

파일 변경은 계산 결과가 아니라 commit protocol이다

read·match·patch·write 사이에서 기준 content가 바뀔 수 있으므로 파일 변경은 사전조건을 가진 낙관적 commit으로 읽어야 한다.

목차

편집 함수가 새 문자열을 계산했다는 사실과 파일이 안전하게 바뀌었다는 사실은 다르다. 기준 content와 현재 content의 동일성, target range의 유일성, write 직전의 검증이 commit 가능성을 결정한다.

A23은 이 간격을 명시적으로 다룬다. 실패 시에는 부분 변경을 성공처럼 보고하지 않아야 하고, 성공 시에도 tool result와 transcript가 같은 commit identity를 참조해야 resume에서 중복 적용을 피할 수 있다.

이 장의 결정점

> **A22 · Read가 edit precondition을 만든다**
>
> FileRead는 출력만 반환하지 않고 이후 Edit가 검증할 file observation state를 남긴다.
>
> `src/tools/FileReadTool/FileReadTool.ts:1020–1040`
>
> **보존 조건.** partial read 범위를 구분한다.

> **A23 · FileEdit optimistic validation과 commit window**
>
> stale check 뒤 write까지 JS event loop에 제어를 넘기지 않는 좁은 동기 구간을 만든다.
>
> `src/tools/FileEditTool/FileEditTool.ts:427–491`
>
> **보존 조건.** check와 synchronous write 사이 내부 await가 없다.

> **A24 · Tool contract characterization**
>
> 리팩터링 전에 각 tool의 관찰 가능한 contract를 자동 추출·고정한다.
>
> `src/Tool.ts:743–791`
>
> **보존 조건.** 실제 effect는 fake adapter로 격리한다.

심층 코드 독해 · A23

FileEdit optimistic validation과 commit window

Edit는 모델이 본 old_string을 현재 파일에 적용하지만
permission·history 같은 await 사이에 외부 변경이 끼어들 수 있다.
commit window를 줄여도 완전한 lock은 아니다.

> **독자 질문.** stale validation과 동기 write 사이의 좁은 window에서 무엇이 보장되고 어떤 race가 남는가?

이 장의 시작 · A23 ATLAS · 목차

결정적인 코드를 읽기 전에

compare-and-swap와 commit window

읽은 version이 아직 current일 때만 새 값을 쓴다는 것이 CAS의 직관이다. 검사와 write가 하나의 filesystem atomic primitive가 아니면 그 사이 race는 남는다.

> **source에서 볼 지점.** 427-491행에서 await를 앞당긴 뒤 stale check부터 sync write까지 await가 없는 이유를 본다.

검사 직후 외부 write가 끼면 optimistic check만으로는 덮어쓰기를 막지 못한다

두 writer가 같은 v1을 읽는 interleaving을 본다.

A23 축소 예제

```
A reads v1
B reads v1
A validates v1
B writes v2
A writes v3 // B의 변경을 덮음
```

지점	관찰	해석
공통 read	A와 B 모두 v1 관찰	둘 다 아직 valid하다고 믿는다.
A validate	현재 v1 확인	atomic commit은 아직 아님
B→A write	v2 뒤 v3	validation과 write 사이 race가 드러난다.

> **프로젝트로 돌아가기.** 실제 구현이 await를 제거해 window를 줄이지만 외부 process를 lock하지는 않는다는 경계를 설명한다.

고정 source에서 결정점을 찾는다

모델이 읽은 old string을 현재 파일에 안전하게 적용하고 외부 변경을 덮어쓰지 않는다.

src/tools/FileEditTool/FileEditTool.ts:427-491 · await를 앞당긴 뒤 stale check→patch→동기 write

```
427      // Ensure parent directory exists before the atomic ↩
read-modify-write section.
428      // These awaits must stay OUTSIDE the critical section ↩
below — a yield between
429      // the staleness check and writeTextContent lets concurrent ↩
edits interleave.
430      await fs.mkdir(dirname(absoluteFilePath))
431      if (fileHistoryEnabled()) {
432        // Backup captures pre-edit content — safe to call before ↩
the staleness
433        // check (idempotent v1 backup keyed on content hash; if ↩
staleness fails
434        // later we just have an unused backup, not corrupt state).
435        await fileHistoryTrackEdit(
436          updateFileHistoryState,
437          absoluteFilePath,
438          parentMessage.uuid,
439        )
440      }
441
442      // 2. Load current state and confirm no changes since last ↩
read
443      // Please avoid async operations between here and writing ↩
to disk to preserve atomicity
444      const {
445        content: originalFileContents,
446        fileExists,
447        encoding,
448        lineEndings: endings,
449      } = readFileForEdit(absoluteFilePath)
450
451      if (fileExists) {
452        const lastWriteTime = getFileModificationTime(absoluteFil ↩
ePath)
453        const lastRead = readFileState.get(absoluteFilePath)
454        if (!lastRead || lastWriteTime > lastRead.timestamp) {
455          // Timestamp indicates modification, but on Windows ↩
timestamps can change
456          // without content changes (cloud sync, antivirus, etc ↩
.). For full reads,
     … 35개 행은 source link에서 이어 읽기
```

고정 revision의 전체 범위 열기

구조 안에서 누가 소유하는가

OBSERVED

FileEditTool이 current content, patched content, history snapshot과 write 순서를 소유한다.

INFERENCE

readFileState가 precondition을 제공하고 FileEdit가 검증·patch·commit을 한 critical section처럼 배열한다.

입력. file_path, old_string, new_string, replace_all · read state

유지 state. current content · patched content · history snapshot

상태가 어떻게 바뀌는가

1. directory/history 준비 같은 async 작업을 먼저 끝낸다.
2. commit 직전 현재 파일을 다시 읽는다.
3. read state와 stale 여부를 검사한다.
4. old string match와 uniqueness를 검증한다.
5. patch를 계산하고 write한다.
6. LSP/history/UI observer를 갱신한다.

긴 lock 대신 commit 직전 version validation을 쓰는 optimistic 전략이다.

보존 조건과 깨지는 방식

- check와 synchronous write 사이 내부 await가 없다.
- old_string mismatch는 write하지 않는다.
- write 뒤 observer가 이전 content를 canonical로 보지 않는다.

반증 가능한 실패

- 외부 process race
- mtime/content token 약함
- observer update 실패
- replace_all 과잉 변경

배경 이론은 source로 돌아오기 위한 도구다

- **optimistic concurrency control** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.
- **critical section** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.
- **compare-and-swap** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.

atomic temp-write/rename과 stronger version token을 검토하고 effect receipt에 before/after digest를 남긴다.

실행과 운영에서 생기는 결과

OBSERVED	permission 등 비동기 작업을 먼저 끝내고 current version을 확인한 뒤 patch와 sync write를 연속 수행한다.
INFERENCE	filesystem write가 commit이며 history/observer update는 후속 관찰 effect다.
UNKNOWN	외부 process race, 약한 token, observer failure, replace_all 과잉 변경은 successful return과 실제 보장을 갈라놓는다.
RECOMMENDATION	before/after digest, token, write receipt, observer disposition을 함께 남겨야 한다.

지금 재구성할 수 있는 시스템 상태

OBSERVED	await를 앞당기고 stale check→patch→동기 write를 좁게 배열한다.
INFERENCE	같은 process event loop의 interleaving은 줄지만 외부 writer가 없다는 보장은 아니다.
UNKNOWN	filesystem atomicity와 외부 editor race 빈도는 source만으로 확정할 수 없다.
RECOMMENDATION	temp-write/rename을 검토할 때 metadata와 symlink semantics, history update 순서도 함께 검증해야 한다.

다음 코드를 읽을 때 가져갈 질문

- write는 성공했지만 observer update가 실패하면 caller에게 어떤 receipt를 반환해야 하는가?
- atomic rename이 content race를 줄여도 permission과 path identity는 왜 별도 문제인가?

IV

transcript, compaction, recovery, nested agent

살아 있는 state와 지속된 event를 구분하고, 손실을 인식한 채 대화를 다시 구성하는 방법을 읽는다.

목차로

장

append-only transcript와 compaction

지속성은 현재 message 배열의 복사본이 아니라, 순서·identity·parent 관계를 가진 event 기록과 projection의 문제다.

목차

transcript는 UI 로그가 아니다. 어떤 message와 tool result가 같은 turn에 속하는지, 중단 뒤 어디부터 재개할지 결정하는 입력이다. append-only 기록은 과거를 덮어쓰지 않지만 그 자체로 valid state를 보장하지 않으므로 scan과 projection 규칙이 필요하다.

compaction은 단순 요약이 아니라 향후 결정을 위해 어떤 정보를 보존할지 선택하는 손실 변환이다. summary와 원 event가 함께 있을 때 어느 쪽이 다음 context의 authority가 되는지 명확해야 한다.

그림 5 · transcript·compaction·recovery에서 이 장의 정적 책임과 동적 연결을 먼저 확인한다.

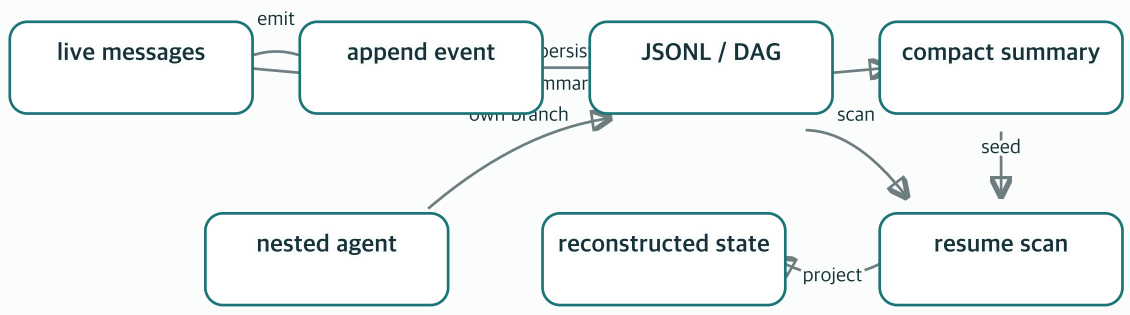


그림 5
transcript·compaction·recovery — 메모리의 message 배열, append-only event, compact summary, resume projection은 서로 다른 손실과 복구 특성을 갖는다. 복구는 단순 재로딩이 아니라 event를 다시 유효한 대화 state로 투영하는 과정이다.

이 장의 결정점

A25 · Leaf-to-root conversation chain reconstruction

resume은 최신 leaf에서 parentUuid를 따라가며 root-to-leaf history를 만든다.

`src/utils/sessionStorage.ts:2069–2117`

보존 조건. 같은 UUID를 두 번 방문하지 않는다.

A26 · 병렬 tool result orphan recovery

single-parent chain이 잃은 sibling assistant와 tool_result를 message.id와 parentUuid index로 되찾는다.

`src/utils/sessionStorage.ts:2118–2205`

보존 조건. 같은 recovered UUID를 중복 삽입하지 않는다.

A27 · Resume consistency telemetry

저장 당시 messageCount와 replay 위치를 비교해 round-trip drift를 측정한다.

`src/utils/sessionStorage.ts:2208–2243`

보존 조건. 가장 최근 checkpoint 하나를 사용한다.

A28 · Cooperative cancellation propagation

하나의 abort intent를 provider, tool, child task가 각 safe point에서 관찰한다.

`src/services/tools/toolExecution.ts:413–468`

보존 조건. abort 후 새 effect를 시작하지 않는다.

A29 · Post-compaction context restoration

요약 뒤에도 최근 file, plan, skills, async agents를 typed attachment로 재주입한다.

`src/services/compact/compact.ts:1399–1448`

보존 조건. preserved content를 중복 주입하지 않는다.

심층 코드 독해 · A25

Leaf-to-root conversation chain reconstruction

transcript 파일은 append 순서대로 읽는 단순 log가 아니라 parentUuid로 branch가 갈리는 graph다. 사용자가 선택한 leaf에서 어떤 대화가 실제 ancestry인지 복원해야 resume context가 정해진다.

독자 질문. leaf에서 parent를 거슬러 올라갈 때 missing edge와 cycle을 어떻게 다루고 sibling을 왜 자동 포함하지 않는가?

이 장의 시작 · A25 ATLAS · 목차

결정적인 코드를 읽기 전에

linked lineage와 cycle detection

각 node가 parent 하나를 가리키면 leaf-to-root walk로 한 계보를 복원할 수 있다. seen set은 malformed cycle에서 무한 loop를 막지만 누락된 branch를 자동 복구하지는 않는다.

> **source에서 볼 지점.** 2069-2117행에서 parent lookup 실패와 seen 처리, 마지막 reverse 순서를 본다.

append 순서와 선택된 lineage는 같지 않다

한 parent에서 두 branch가 갈라진 transcript를 축소한다.

A25 축소 예제

```
root <- a <- leafA
        \ b <- leafB
selected = leafB
chain = [root, b, leafB] // a, leafA는 파일에 있어도 제외
```

지점	관찰	해석
map 구성	모든 UUID record가 존재	저장된 집합과 선택 context는 다르다.
leafB walk	parent를 따라 b→root	하나의 ancestry만 얻는다.
reverse	root→b→leafB	provider에 시간 순 context를 제공한다.

> **프로젝트로 돌아가기.** 실제 함수가 file order가 아니라 parent relation을 authority로 삼는 이유다.

고정 source에서 결정점을 찾는다

한 파일의 여러 branch/metadata 중 선택된 conversation lineage를 복원한다.

src/utils/sessionStorage.ts:2069-2117 · buildConversationChain

```
2069  export function buildConversationChain(
2070    messages: Map<UUID, TranscriptMessage>,
```

```
2071    leafMessage: TranscriptMessage,
2072  ): TranscriptMessage[] {
2073    const transcript: TranscriptMessage[] = []
2074    const seen = new Set<UUID>()
2075    let currentMsg: TranscriptMessage | undefined = leafMessage
2076    while (currentMsg) {
2077      if (seen.has(currentMsg.uuid)) {
2078        logError(
2079          new Error(
2080            `Cycle detected in parentUuid chain at message ↩
 ${currentMsg.uuid}. Returning partial transcript.`,
2081          ),
2082        )
2083        logEvent('tengu_chain_parent_cycle', {})
2084        break
2085      }
2086      seen.add(currentMsg.uuid)
2087      transcript.push(currentMsg)
2088      currentMsg = currentMsg.parentUuid
2089        ? messages.get(currentMsg.parentUuid)
2090        : undefined
2091    }
2092    transcript.reverse()
2093    return recoverOrphanedParallelToolResults(messages, ↩
 transcript, seen)
2094  }
2095
2096  /**
2097   * Post-pass for buildConversationChain: recover sibling ↩
 assistant blocks and
2098   * tool_results that the single-parent walk orphaned.
      … 19개 행은 source link에서 이어 읽기
```

고정 revision의 전체 범위 열기

구조 안에서 누가 소유하는가

OBSERVED	buildConversationChain이 UUID map, 선택 leaf, seen set, 역방향 walk 결과를 소유한다.
INFERENCE	storage는 모든 record를 보존하고 chain builder가 한 resume view의 canonical lineage를 선택한다.

입력. UUID→TranscriptMessage map · 선택된 leaf

유지 state. transcript array · seen UUID set · current message

상태가 어떻게 바뀌는가

1. leaf를 transcript에 넣는다.

2. parentUuid로 이전 message를 찾는다.
3. cycle이면 partial chain과 telemetry를 남기고 중단한다.
4. root에 도달하면 reverse한다.
5. 병렬 orphan recovery pass를 실행한다.

append record를 직접 정렬하기보다 explicit parent relation을 canonical lineage로 사용한다.

보존 조건과 깨지는 방식

- 같은 UUID를 두 번 방문하지 않는다.
- 결과 순서는 root→leaf다.
- broken parent를 조용히 다른 branch로 대체하지 않는다.

반증 가능한 실패

- missing parent
- cycle
- 병렬 sibling이 single-parent walk에서 탈락

배경 이론은 source로 돌아오기 위한 도구다

- **linked lineage** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.
- **cycle detection** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.
- **event replay** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.

replay result에 repair ledger와 missing edge를 함께 반환한다.

실행과 운영에서 생기는 결과

OBSERVED	selected leaf에서 parentUuid를 따라가 seen set에 넣고 root 도달 뒤 뒤집는다.
INFERENCE	복원된 chain이 다음 model context가 되므로 제외된 branch는 resume에서 보이지 않는다.
UNKNOWN	missing parent, cycle, 병렬 sibling은 서로 다른 repair 필요성을 가진다.

RECOMMENDATION	selected leaf, missing edge, cycle stop, excluded sibling count를 repair ledger로 남겨야 한다.

지금 재구성할 수 있는 시스템 상태

OBSERVED	UUID map과 seen set으로 leaf-to-root chain을 만든다.
INFERENCE	single-parent link가 온전한 lineage에서는 canonical resume order를 얻는다.
UNKNOWN	저장 중 부분 write가 만든 모든 graph 손상 패턴과 복구 정책은 알 수 없다.
RECOMMENDATION	A26 orphan recovery가 기본 chain 밖의 어떤 병렬 result만 선택적으로 삽입하는지 이어서 본다.

다음 코드를 읽을 때 가져갈 질문

- 선택 leaf의 parent가 없으면 root로 간주할 것인가 불완전 chain으로 표시할 것인가?
- 두 sibling branch를 모두 합치면 사용자 대화의 causal order가 왜 달라질 수 있는가?

장

오류, 중단, resume의 재구성

실패는 throw 한 지점보다 partial observation과 durable commit 사이의 위치로 분류해야 한다.

목차

같은 오류라도 provider event를 아직 내보내지 않은 경우, UI에는 보였지만 canonical message가 닫히지 않은 경우, tool effect는 끝났지만 transcript append가 실패한 경우의 복구 전략은 다르다. terminal reason만으로 이 차이를 표현할 수 없다.

A41은 resume 시 event를 읽어 현재 state로 투영하는 경계를 보여 준다. 이 과정은 마지막 줄을 읽는 것이 아니라 parent/turn/tool pairing을 복원하고, 불완전 tail을 어떤 상태로 취급할지 정하는 알고리즘이다.

이 장의 결정점

> **A13 · Compaction as lossy migration**
>
> 요약은 compression이 아니라 future action을 위한 상태 재구성이다.
>
> `src/services/compact/compact.ts:596–742`
>
> **보존 조건.** tool_use identity와 unresolved work를 자연어 요약만으로 대체하지 않는다.

> **A40 · Session memory extraction trigger**
>
> token growth, tool-call 수, 자연스러운 turn 경계를 함께 사용해 비싼 memory extraction의 시점을 결정한다.
>
> `src/services/SessionMemory/sessionMemory.ts:134–179`
>
> **보존 조건.** token growth threshold는 두 trigger 경로 모두에서 반드시 필요하다.

> **A41 · Capability가 축소된 memory extraction agent**
>
> parent context를 복제하지 않고 별도 context와 “정확한 memory file 하나의 Edit만” 허용하는 canUseTool로 보조 agent 권한을 줄인다.
>
> `src/services/SessionMemory/sessionMemory.ts:272–337`
>
> **보존 조건.** subagent와 teammate query에서는 자동 extraction을 실행하지 않는다.

Capability가 축소된 memory extraction agent

memory 품질을 위해 agent runtime을 재사용하지만 보조 agent가 workspace 전체와 parent context를 바꿀 권한은 없다. 같은 runtime이라도 capability와 state handoff를 축소해야 한다.

> **독자 질문.** sequential lock, isolated context, restricted read state가 실제로 어떤 권한과 race를 제거하는가?

이 장의 시작 · A41 ATLAS · 목차

결정적인 코드를 읽기 전에

capability attenuation과 구조화된 task lifetime

child에는 parent보다 좁은 tool·path·context만 전달하고, parent는 child 완료 또는 중단을 정산한다. callback 하나를 제한해도 다른 inherited capability가 남으면 blast radius는 줄지 않는다.

> **source에서 볼 지점.** 272-337행에서 fork 시 복사되는 context와 새로 제한되는 callback을 목록으로 대조한다.

경로 문자열 제한만으로는 같은 파일 identity를 보장하지 못한다

허용 path가 symlink를 통해 다른 target을 가리키는 경우를 본다.

A41 축소 예제

```
allowed = '/workspace/memory.md'
// memory.md -> /outside/shared.md (symlink)
if (request.path === allowed) write(request.path)
// 문자열은 같지만 effect identity는 다를 수 있음
```

지점	관찰	해석
문자열 검사	요청 path가 allowed와 같음	표면적 capability는 통과한다.
filesystem resolve	target은 outside file	실제 effect boundary가 넓어진다.
receipt	realpath/digest가 없으면	어느 resource가 바뀌었는지 복원하기 어렵다.

> **프로젝트로 돌아가기.** restricted callback뿐 아니라 realpath identity와 inherited tool set을 함께 검토해야 한다.

고정 source에서 결정점을 찾는다

model 기반 summarization이 필요하지만 보조 실행이 workspace의 다른 파일이나 parent state를 임의로 바꾸지 못하게 한다.

src/services/SessionMemory/sessionMemory.ts:272-337 · sequential extraction과 forked agent

```
272  const extractSessionMemory = sequential(async function (
273    context: REPLHookContext,
274  ): Promise<void> {
275    const { messages, toolUseContext, querySource } = context
276
277    // Only run session memory on main REPL thread
278    if (querySource !== 'repl_main_thread') {
279      // Don't log this - it's expected for subagents, teammates, ↩
etc.
280      return
281    }
282
283    // Check gate lazily when hook runs (cached, non-blocking)
284    if (!isSessionMemoryGateEnabled()) {
285      // Log gate failure once per session (ant-only)
286      if (process.env.USER_TYPE === 'ant' && ↩
!hasLoggedGateFailure) {
287        hasLoggedGateFailure = true
288        logEvent('tengu_session_memory_gate_disabled', {})
289      }
290      return
291    }
292
293    // Initialize config from remote (lazy, only once)
294    initSessionMemoryConfigIfNeeded()
295
296    if (!shouldExtractMemory(messages)) {
297      return
298    }
299
300    markExtractionStarted()
301
     … 36개 행은 source link에서 이어 읽기
```

고정 revision의 전체 범위 열기

구조 안에서 누가 소유하는가

OBSERVED	SessionMemory extraction이 sequential lock, isolated setupContext, restricted readFileState와 in-progress marker를 소유한다.
INFERENCE	parent REPL은 추출 요청과 결과 handoff를 소유하고 forked agent는 memory 범위의 계산만 소유한다.

입력. REPLHookContext · currentMemory · memory path · threshold state

유지 state. sequential extraction lock · isolated setupContext · restricted readFileState · extraction-in-progress marker

상태가 어떻게 바뀌는가

1. main REPL thread와 feature gate를 확인한다.
2. threshold가 충족되면 extraction started를 기록한다.
3. parent ToolUseContext에서 격리된 subagent context를 만든다.
4. 현재 memory를 읽고 update prompt를 만든다.
5. forked agent에 cache-safe params와 restricted canUseTool을 전달한다.
6. usage와 threshold config를 기록하고 token 기준점을 갱신한다.
7. finally에서 extraction marker를 정리한다.

요약 품질을 위해 agent runtime을 재사용하되 capability attenuation과 context isolation으로 blast radius를 제한한다.

보존 조건과 깨지는 방식

- subagent와 teammate query에서는 자동 extraction을 실행하지 않는다.
- 한 번에 하나의 extraction만 sequential wrapper를 통과한다.
- forked agent는 지정된 path에 대한 Edit 이외의 tool effect를 얻지 못한다.
- 실패해도 extraction-in-progress marker는 terminal 상태로 돌아간다.

반증 가능한 실패

- path 문자열은 같지만 symlink/realpath identity가 다름
- restricted callback 밖의 inherited capability
- agent가 edit 없이 성공 종료

- 15초 waiter timeout 뒤 compaction과 extraction이 경쟁

배경 이론은 source로 돌아오기 위한 도구다

- **capability attenuation** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.
- **structured task lifecycle** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.
- **state handoff** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.

memory path를 realpath/file identity로 고정하고 extraction result에 before/after digest와 last summarized UUID를 원자적으로 기록한다.

실행과 운영에서 생기는 결과

OBSERVED	추출을 직렬화하고 축소한 context로 child agent를 실행해 memory result를 받아 쓴다.
INFERENCE	model call과 memory file write가 발생하며 timeout 뒤에도 child가 settle되지 않았을 수 있다.
UNKNOWN	symlink identity, inherited capability, edit 없는 성공, waiter timeout 경쟁은 parent가 잘못 완료를 판단하게 한다.
RECOMMENDATION	authority set, child task id, before/after digest, last summarized UUID, settlement를 receipt로 남겨야 한다.

지금 재구성할 수 있는 시스템 상태

OBSERVED	sequential extraction과 forked agent, 제한된 read state가 구현돼 있다.
INFERENCE	제한되지 않은 다른 capability가 없고 path가 실제 file identity와 일치할 때 blast radius가 좁다.
UNKNOWN	feature variant의 전체 inherited tool set과 timeout 후 실제 settlement는 확인되지 않는다.
RECOMMENDATION	realpath/file identity와 atomic write receipt를 추가하기 전 fork context 전체를 inventory해야 한다.

다음 코드를 읽을 때 가져갈 질문

- child가 성공 종료했지만 memory file digest가 같다면 완료로 볼 수 있는가?
- 15초 waiter가 끝난 뒤 compaction이 시작되면 extraction과 어떤 state를 경쟁하는가?

장

nested agent와 독립된 lifecycle

하위 agent는 함수 호출이 아니라 별도 context·effect·result를 가진 lifecycle이며 parent와의 결합 지점이 명시되어야 한다.

목차

nested agent를 병렬 함수처럼 취급하면 취소, budget, transcript branch, tool authority가 사라진다. parent는 objective와 제한을 전달하지만 하위 실행의 모든 중간 state를 직접 소유하지 않는다.

A30은 생성과 결과 수집 사이의 identity를, A31은 lifecycle 정리와 실패 전파를 드러낸다. 하위 결과를 parent message로 투영하는 순간에는 provenance와 terminal reason이 함께 돌아와야 한다.

이 장의 결정점

> **A30 · Child agent runtime spawn과 event projection**
>
> AgentTool은 parent context를 투영해 별도의 query loop를 열고 bounded result를 parent로 돌려준다.
>
> `src/tools/AgentTool/runAgent.ts:500–770`
>
> **보존 조건.** child authority가 parent보다 조용히 넓어지지 않는다.

> **A31 · Incomplete child tool call filtering**
>
> child 결과를 합류할 때 unresolved tool_use를 그대로 parent context에 노출하지 않는다.
>
> `src/tools/AgentTool/runAgent.ts:866–905`
>
> **보존 조건.** resolved pair는 유지한다.

심층 코드 독해 · A30

Child agent runtime spawn과 event projection

긴 하위 작업을 child agent에 넘기면 parent context는 작아지지만 child effect와 transcript, cancellation이 새로운 nested runtime을 만든다.

> **독자 질문.** parent authority를 줄여 전달하면서 child progress/result와 settlement를 어떤 identity로 다시 합치는가?

이 장의 시작 · A30 ATLAS · 목차

결정적인 코드를 읽기 전에

중첩 상태기계와 structured task lifecycle

parent의 한 tool state 안에 child의 여러 turn 상태가 들어간다. parent terminal은 child가 완료·실패·취소로 settle되고 결과가 projection된 뒤여야 한다.

> **source에서 볼 지점.** 500-770행에서 controller, context, transcript, query event가 어느 task id로 합류하는지 본다.

고정 source에서 결정점을 찾는다

복잡한 하위 작업을 독립 context와 tool set으로 실행하면서 parent authority와 cancellation을 보존한다.

src/tools/AgentTool/runAgent.ts:500-770 · child controller·context·transcript·query 합류

```
500    const resolvedTools = useExactTools
501      ? availableTools
502      : resolveAgentTools(agentDefinition, availableTools, ↩
isAsync).resolvedTools
503
504    const additionalWorkingDirectories = Array.from(
505      appState.toolPermissionContext.additionalWorkingDirectories ↩
.keys(),
506    )
507
508    const agentSystemPrompt = override?.systemPrompt
509      ? override.systemPrompt
510      : asSystemPrompt(
511          await getAgentSystemPrompt(
512            agentDefinition,
513            toolUseContext,
514            resolvedAgentModel,
515            additionalWorkingDirectories,
516            resolvedTools,
517          ),
518        )
519
520    // Determine abortController:
521    // - Override takes precedence
522    // - Async agents get a new unlinked controller (runs ↩
independently)
523    // - Sync agents share parent's controller
524    const agentAbortController = override?.abortController
525      ? override.abortController
526      : isAsync
527        ? new AbortController()
528        : toolUseContext.abortController
529
     … 241개 행은 source link에서 이어 읽기
```

고정 revision의 전체 범위 열기

구조 안에서 누가 소유하는가

OBSERVED	runAgent가 child messages, abort/task identity, progress/result와 child transcript를 소유한다.
INFERENCE	parent tool call이 spawn authority를 소유하고 child runtime이 격리된 query/tool loop를 소유한다.

입력. agent definition · prompt · parent ToolUseContext · model/tools/budget

유지 state. child messages · child abort/task identity · progress/result

상태가 어떻게 바뀌는가

1. agent config와 MCP/tool set을 준비한다.
2. parent context에서 허용된 state를 child로 투영한다.
3. child query()를 실행한다.
4. recordable message를 sidechain에 기록한다.
5. progress를 parent task/UI에 투영한다.
6. terminal summary/result를 합류시킨다.

parent context를 오염시키지 않고 긴 하위 탐색을 격리한다.

보존 조건과 깨지는 방식

- child authority가 parent보다 조용히 넓어지지 않는다.
- child message history와 parent summary를 구분한다.
- cancel 시 child task와 signal이 수렴한다.

반증 가능한 실패

- context 과다 복사
- child effect는 commit됐지만 parent result 누락
- sidechain과 task status drift

배경 이론은 source로 돌아오기 위한 도구다

- **nested state machine** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.
- **capability attenuation** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.
- **structured task lifecycle** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.

spawn receipt에 authority, budget, source revision, settlement condition을 명시한다.

실행과 운영에서 생기는 결과

OBSERVED	agent definition과 축소한 context로 child controller를 만들고 query event를 parent progress/result로 투영한다.
INFERENCE	child는 허용된 tool로 외부 effect를 만들고 별도 transcript를 남길 수 있다.

UNKNOWN	context 과다 복사, committed effect 뒤 parent result 누락, status drift는 parent가 재시도하게 만들 수 있다.
RECOMMENDATION	spawn authority/budget/revision, child task id, effect receipts, settlement reason을 묶어야 한다.

지금 재구성할 수 있는 시스템 상태

OBSERVED	child controller·context·transcript·query를 조립하는 runtime path가 있다.
INFERENCE	전달 capability와 cancellation이 모든 child executor에 적용될 때 격리가 유지된다.
UNKNOWN	외부 effect의 compensation과 모든 sidechain/status 동기화는 이 범위로 확정되지 않는다.
RECOMMENDATION	spawn receipt와 settlement condition을 명시하기 전 parent/child status writer를 대조해야 한다.

다음 코드를 읽을 때 가져갈 질문

- child effect는 commit됐지만 parent result가 끊기면 같은 AgentTool 재시도를 어떻게 식별할 것인가?
- parent abort 뒤 child transcript의 마지막 partial message는 어디까지 보존해야 하는가?

V

MCP, plugin, remote, config와 운영의 변형 경계

확장 기능을 core의 목록이 아니라 발견·구성·transport·control plane의 책임 분리로 읽는다.

목차로

장

MCP와 plugin registry

확장은 발견된 항목을 즉시 신뢰하는 일이 아니라 identity, capability, configuration을 composition에 안전하게 편입하는 일이다.

목차

MCP와 plugin은 모두 surface를 늘리지만 같은 경계가 아니다. MCP는 외부 server의 tool/resource와 transport를 연결하고, plugin은 로컬 발견·설정·명령 또는 hook surface를 composition에 추가한다. 각 경로는 이름 충돌, stale config, unavailable endpoint를 별도로 처리해야 한다.

A44는 registry merge에서 우선순위와 충돌을 다룬다. 단순 object spread는 마지막 값이 이긴다는 결과만 만들 뿐, 독자가 충돌 이유와 실제 owner를 추적할 수 있는 provenance를 남기지 않는다.

그림 6 · 확장과 control plane에서 이 장의 정적 책임과 동적 연결을 먼저 확인한다.

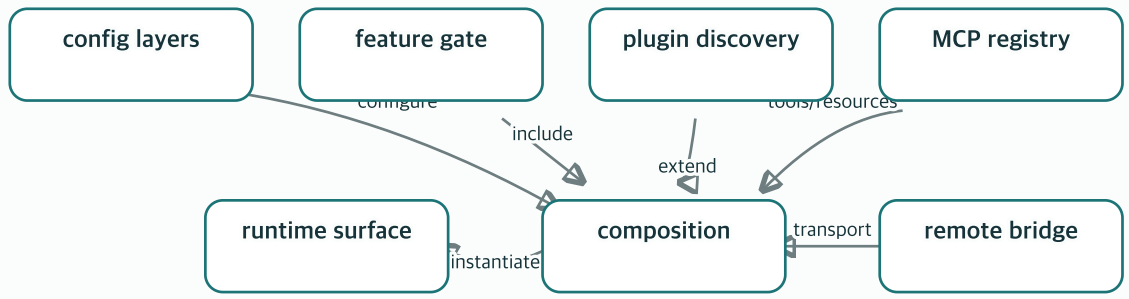


그림 6
확장과 control plane — MCP·plugin·remote·config는 core를 대체하지 않고 구성과 transport, capability surface를 변화시킨다. build-time gate와 runtime policy를 한 축으로 합치면 실제 후보 경계를 오독한다.

이 장의 결정점

A32 · MCP server name normalization

외부 이름을 API-safe identity로 바꾸되 delimiter 충돌을 줄인다.

`src/services/mcp/normalization.ts:1–23`

보존 조건. 결과는 API name pattern을 만족한다.

A33 · Background plugin installation queue

plugin install은 UI blocking을 피하면서 marketplace 상태와 실패를 조정한다.

`src/services/plugins/PluginInstallationManager.ts:60–120`

보존 조건. 한 plugin의 실패가 상태를 영원히 installing으로 남기지 않는다.

A44 · WebSocket close code별 bounded reconnect

연결 종료를 permanent, session-not-found transient, ordinary transient로 분류하고 서로 다른 retry budget으로 terminal 여부를 결정한다.

`src/remote/SessionsWebSocket.ts:208–300`

보존 조건. permanent close는 reconnect하지 않는다.

심층 코드 독해 · A44

WebSocket close code별 bounded reconnect

remote session의 network는 일시적으로 끊길 수 있지만 server가 session 없음이나 terminal close를 확정했다면 무한 reconnect는 잘못이다. close code와 attempt budget이 다음 상태를 결정한다.

> **독자 질문.** manual close, transient close, session-not-found, token refresh를 어떤 generation과 budget으로 구분하는가?

이 장의 시작 · A44 ATLAS · 목차

결정적인 코드를 읽기 전에

connection state machine과 bounded retry

open/closing/closed/reconnecting은 boolean 하나로 표현하기 어렵다. close reason마다 terminal 여부와 별도 retry budget을 두어 availability와 retry storm을 절충한다.

> **source에서 볼 지점.** 208-300행에서 close code 분기, counter 증가/reset, timer 설치와 manual reconnect를 함께 본다.

늦은 reconnect timer가 새 connection generation을 덮지 않아야 한다

첫 socket의 timer가 예약된 뒤 사용자가 수동 reconnect하는 interleaving이다.

A44 축소 예제

```
socket1 closes -> timer1 scheduled
manualReconnect() -> socket2 opens
timer1 fires -> must not replace socket2
// generation1 timer는 generation2 state를 쓸 권한이 없음
```

지점	관찰	해석
close 1	transient로 timer1 예약	generation1의 복구 작업이다.
manual reconnect	socket2 연결	현재 owner가 generation2로 바뀐다.
timer1 fire	stale callback	generation check 없으면 healthy socket2를 교체한다.

> **프로젝트로 돌아가기.** timer callback이 현재 ws/state를 검증하는지와 counter reset 시점을 실제 코드에서 본다.

고정 source에서 결정점을 찾는다

remote session의 일시적 network 단절을 복구하면서 server가 확정한 종료나 존재하지 않는 session을 무한 재시도하지 않는다.

src/remote/SessionsWebSocket.ts:208-300 · message validation과 reconnect state machine

```
208     * Handle incoming WebSocket message
209     */
210    private handleMessage(data: string): void {
211      try {
212        const message: unknown = jsonParse(data)
213
214        // Forward SDK messages to callback
215        if (isSessionsMessage(message)) {
216          this.callbacks.onMessage(message)
217        } else {
218          logForDebugging(
219            `[SessionsWebSocket] Ignoring message type: ${typeof ↩
message === 'object' && message !== null && 'type' in message ? ↩
String(message.type) : 'unknown'}`,
220          )
221        }
222      } catch (error) {
223        logError(
224          new Error(
225            `[SessionsWebSocket] Failed to parse message: ↩
${errorMessage(error)}`,
226          ),
227        )
228      }
229    }
230
231    /**
232     * Handle WebSocket close
233     */
234    private handleClose(closeCode: number): void {
235      this.stopPingInterval()
236
237      if (this.state === 'closed') {
     … 63개 행은 source link에서 이어 읽기
```

고정 `revision`의 전체 범위 열기

구조 안에서 누가 소유하는가

OBSERVED	SessionsWebSocket이 ws handle, connection state, reconnect attempts, session-not-found retries, timer를 소유한다.
INFERENCE	transport adapter가 socket lifetime과 재접속을 소유하고 상위 session은 remote event의 제품 의미를 소유한다.

입력. 현재 WebSocketState · close code · reconnect attempt counters · retry constants

유지 state. ws handle · state · reconnectAttempts · sessionNotFoundRetries · reconnectTimer

상태가 어떻게 바뀌는가

1. 수신 JSON을 parse하고 SessionsMessage runtime guard를 통과한 값만 callback에 전달한다.
2. close에서 ping을 멈추고 기존 state를 보존한 뒤 handle을 해제한다.
3. permanent close code면 즉시 terminal callback을 호출한다.
4. 4001이면 compaction 중 transient 가능성을 인정하되 전용 retry budget과 증가 delay를 적용한다.
5. 일반 close는 이전에 connected였고 budget이 남을 때만 reconnect를 예약한다.
6. timer가 다시 fresh token으로 connect를 시도한다.

availability를 높이되 server terminal semantics와 retry storm 방지를 우선하는 explicit connection state machine을 둔다.

보존 조건과 깨지는 방식

- permanent close는 reconnect하지 않는다.
- 동시에 하나의 reconnect timer만 다음 연결을 소유한다.
- 처음부터 연결되지 못한 상태와 연결 뒤 끊긴 상태를 같은 retry로 처리하지 않는다.
- unknown/invalid message는 application callback으로 승격하지 않는다.

반증 가능한 실패

- close와 manual reconnect timer race
- 4001 의미가 server version에서 달라짐
- token refresh와 reconnect ordering
- 연결 복구 뒤 event 중복 또는 gap

배경 이론은 source로 돌아오기 위한 도구다

- **state machine** — 현재 상태와 입력이 허용된 다음 상태와 action을 결정하는 모델이다.
- **retry budget** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.
- **anti-corruption** — 이 결정점을 다른 시스템에서도 비교하기 위한 이론적 어휘다.

connection generation과 last received event identity를 handshake에 포함하고 reconnect 뒤 replay/duplication disposition을 명시한다.

실행과 운영에서 생기는 결과

OBSERVED	message를 validate하고 close code별 terminal/retry 결정을 내린 뒤 bounded delay로 새 socket을 만든다.
INFERENCE	network connection과 event replay가 재시작되며 중복 또는 gap 가능성이 생긴다.
UNKNOWN	timer race, server close-code 의미 drift, token ordering, replay disposition 부재는 연결은 복구돼도 session state를 틀리게 한다.
RECOMMENDATION	connection generation, close code, attempt budget, last event id, replay outcome을 남겨야 한다.

지금 재구성할 수 있는 시스템 상태

OBSERVED	명시적 WebSocketState, close-code 분기, retry counters와 timer가 있다.
INFERENCE	server와 client가 close semantics를 공유하고 stale timer가 current generation을 검사할 때 bounded reconnect가 안전하다.
UNKNOWN	server version별 4001 의미와 reconnect 이후 replay/duplication 계약은 clone만으로 확정할 수 없다.
RECOMMENDATION	handshake에 generation과 last event identity를 넣기 전 server protocol 문서가 admitted되지 않았으므로 해당 claim은 UNKNOWN으로 유지한다.

다음 코드를 읽을 때 가져갈 질문

- session-not-found budget이 끝난 뒤 token refresh가 성공하면 어떤 state에서 새 시도를 허용할 것인가?
- 재연결 뒤 첫 event가 이미 본 id라면 UI와 transcript는 각각 어떻게 deduplicate해야 하는가?

장

remote와 bridge의 transport 경계

같은 turn semantics를 원격으로 옮길 때 transport 재시도와 application 재시도를 구분하지 않으면 effect가 중복될 수 있다.

목차

remote path는 로컬 query를 네트워크로 감싼 것처럼 보여도 연결 identity, serialization, reconnect, cancellation이라는 새 상태를 만든다. transport가 message를 다시 보냈는지와 application이 같은 tool을 다시 실행했는지는 다른 질문이다.

이 snapshot으로 server protocol 전체를 확정할 수 없으므로, 관찰 가능한 client·bridge 경계만 설명하고 서버 측 정산은 UNKNOWN으로 남긴다. 이 제한이 있어야 remote의 존재를 production 운영 보장으로 과장하지 않는다.

이 장의 결정점

> **A34 · Settings precedence resolution**
>
> 값 병합은 source authority와 provenance를 함께 다뤄야 한다.
>
> `src/utils/settings/constants.ts:3–22`
>
> **보존 조건.** later override 규칙이 일관된다.

> **A35 · Remote managed settings checksum과 polling**
>
> 원격 policy는 fetch·auth·checksum·disk cache·background poll을 가진 control plane이다.
>
> `src/services/remoteManagedSettings/index.ts:105–155`
>
> **보존 조건.** 동일 settings는 동일 checksum을 낸다.

> **A36 · Idempotent migration**
>
> durable 설정은 새 코드가 오래된 key와 model alias를 반복 실행 안전하게 변환해야 한다.
>
> `src/migrations/migrateLegacyOpusToCurrent.ts:1–57`
>
> **보존 조건.** 반복 실행 결과가 같다.

configuration과 operations를 읽는 시간축

build-time 제거, startup 구성, runtime policy, telemetry 관찰은 서로 다른 시간에 시스템을 바꾼다.

목차

feature gate를 하나의 on/off로 부르면 bundle에 포함되지 않은 코드와 실행 중 거부된 기능을 구분할 수 없다. config layer의 우선순위와 값의 출처, 적용 시점을 함께 기록해야 재현 가능한 현재 상태가 된다.

telemetry 호출의 존재는 backend 저장이나 실제 대시보드 운영을 증명하지 않는다. 이 책은 event 생성과 local 호출 지점까지만 OBSERVED로 다루고, 수집 이후의 시스템은 UNKNOWN으로 유지한다.

이 장의 결정점

> **A37 · Analytics fan-out과 sink isolation**
>
> event producer는 sink 목록에 기록하고 sink 실패를 product flow와 분리한다.
>
> `src/services/analytics/index.ts:72–165`
>
> **보존 조건.** telemetry 실패가 core effect를 바꾸지 않는다.

> **A38 · Batched exporter flush**
>
> event를 buffer하고 size/time/shutdown 경계에서 전송해 latency와 신뢰성을 절충한다.
>
> `src/services/analytics/datadog.ts:85–130`
>
> **보존 조건.** 동시에 같은 record를 두 batch가 소유하지 않는다.

결정적 부분을 보장하는 구조를 어떻게 읽을 것인가

관찰된 강점

- 진입·turn·stream·effect·durability·extension에 명시적 결정 지점이 존재해 부분 상태를 국소화할 수 있다.
- dynamic import, generator event, tagged union, transcript event처럼 언어·자료구조 메커니즘이 실제 운영 경계와 연결된다.
- permission과 file commit, resume projection을 별도 알고리즘으로 읽을 수 있어 실패 위치를 세분화할 수 있다.

구조적 긴장

- 관찰 event, UI projection, canonical message가 시간차를 두고 공존해 identity와 terminal 의미가 쉽게 흐려진다.
- 명시적 branch는 읽기 쉽지만 route·config·registry가 늘수록 precedence 지식이 여러 파일에 분산된다.
- 복원 snapshot은 넓은 구조를 보여 주지만 upstream intent와 production activation을 끝까지 제한한다.

권고

- 핵심 전이를 reducer나 transition table로 가시화하되 기존 event identity와 effect commit 경계를 먼저 characterization test로 고정한다.
- registry merge, permission decision, remote retry에는 provenance와 idempotency identity를 일급 값으로 둔다.
- UI·transcript·telemetry가 소비하는 projection마다 terminal 조건과 허용된 불완전 상태를 문서화한다.

UNKNOWN

- 공식 source history와 누락 모듈이 보여 줄 다른 구조
- 배포 variant의 feature literal과 production traffic 분포
- 서버 protocol, telemetry backend, 외부 effect의 실제 settlement

변경 영향

새 기능을 평가할 때는 어느 part에 파일을 추가했는지가 아니라 route 후보, state owner, effect authority, durable projection, extension precedence 중 무엇을 바꾸는지 먼저 묻는다. 이 질문에 답하면 영향을 받을 알고리즘과 검증 위치를 atlas와 source index에서 역추적할 수 있다.

목차로 돌아가기

부록 A

A01–A44 알고리즘 atlas

본문은 대표 결정점을 깊게 읽고, atlas는 모든 알고리즘을 source·보존 조건·본문 장으로 다시 찾게 한다.

A01 · 복사본 변환과 반복 stub 생성

복원 builder는 원본을 실행 가능하게 증명하는 compiler가 아니라 실패를 줄여 가는 best-effort reconstruction loop다.

보존 조건. 원본 src는 build.mjs 경로에서 수정하지 않는다.

`scripts/build.mjs:52–117`

본문 장

A02 · 원본을 직접 고치는 prepare-src

별도 build 경로와 달리 prepare-src는 source tree 자체를 rewrite한다.

보존 조건. 재실행은 대부분 동일 결과를 내야 한다.

`scripts/prepare-src.mjs:36–76`

본문 장

A03 · 우선순위 기반 CLI route resolution

entrypoint는 ordered early-return dispatcher다.

보존 조건. 하나의 실행에서 정확히 한 top-level route만 선택된다.

`src/entrypoints/cli.tsx:28–93`

본문 장 · 심층 독해

A04 · ask()의 one-shot engine adapter

호환 wrapper는 외부 mutable state를 새 QueryEngine에 주입하고 결과 state를 다시 꺼낸다.

보존 조건. 한 ask 호출에는 한 engine instance가 있다.

`src/QueryEngine.ts:1249–1295`

본문 장

A05 · print loop의 turn 간 상태 handoff

print는 engine을 유지하지 않고 mutableMessages와 readFileState를 loop 바깥에서 보존한다.

보존 조건. 새 turn은 이전 message history를 본다.

`src/cli/print.ts:2140–2188`

본문 장

A06 · state lifetime classification

리팩터링 전 각 state를 process·session·turn·tool-call·UI-frame 수명으로 분류한다.

보존 조건. 모든 mutable field에는 한 primary owner가 있다.

`src/QueryEngine.ts:184–207`

본문 장

A07 · 사용자 입력 dispatch

입력은 slash command, bash mode, image, normal prompt를 구별하는 작은 language parser다.

보존 조건. local-only output을 provider input과 구분한다.

`src/utils/processUserInput/processUserInput.ts:515–588`

본문 장

A08 · System/User/System context 조립

prompt는 정적 문자열이 아니라 ordered dependency projection이다.

보존 조건. 순서가 바뀌면 instruction scope와 cache prefix가 달라질 수 있다.

`src/utils/queryContext.ts:44–87`

본문 장

A09 · API message normalization

내부 history를 provider가 수용하는 role/content pairing으로 낮춘다.

보존 조건. 각 tool_result는 대응 tool_use 뒤에 온다.

`src/utils/messages.ts:1989–2028`

본문 장

A10 · queryLoop 상태 전이

무한 while보다 중요한 것은 continue마다 새 State를 만드는 전이 규칙이다.

보존 조건. tool_use/tool_result pairing 없이 다음 provider call로 가지 않는다.

`src/query.ts:252–321`

본문 장 · 심층 독해

A11 · tool-use/tool-result pairing

외부 effect는 둘 사이에서 일어나지만 대화 protocol에는 같은 id로 닫힌 원자가 필요하다.

보존 조건. unknown tool도 silent drop하지 않는다.

`src/services/tools/toolExecution.ts:337–410`

본문 장

A39 · 부분 content block의 typed reconstruction

provider event sequence를 index별 mutable builder에 누적하고 block completion에서 immutable-looking AssistantMessage로 투영한다.

보존 조건. delta index에는 먼저 시작된 block이 존재한다.

`src/services/api/claude.ts:1940–2054`

본문 장 · 심층 독해

A12 · API retry와 backoff

retry 여부는 status code뿐 아니라 source, fast mode, retry-after, consumer-visible event에 좌우된다.

보존 조건. abort는 새 retry를 시작하지 않는다.

`src/services/api/withRetry.ts:170–220`

본문 장

A13 · Compaction as lossy migration

요약은 compression이 아니라 future action을 위한 상태 재구성이다.

보존 조건. tool_use identity와 unresolved work를 자연어 요약만으로 대체하지 않는다.

`src/services/compact/compact.ts:596–742`

본문 장

A14 · Message normalization lookup 구축

반복적인 sibling/tool 조회를 index로 바꿔 대화 history를 선형에 가깝게 처리한다.

보존 조건. index는 정확히 같은 immutable message snapshot에 대응한다.

`src/utils/messages.ts:1170–1215`

본문 장

A15 · Generic factory의 default 합성

buildTool은 runtime object spread와 type-level spread를 맞춘다.

보존 조건. def가 default를 덮어쓴다.

`src/Tool.ts:721–791`

본문 장

A16 · 입력별 concurrency batch partition

병렬성은 tool 이름이 아니라 parse된 구체 input의 commutativity를 근사한다.

보존 조건. parse 실패는 unsafe다.

`src/services/tools/toolOrchestration.ts:84–116`

본문 장

A17 · 병렬 결과의 deterministic context merge

병렬 call 중 context mutation을 즉시 적용하지 않고 tool order대로 지연 합성한다.

보존 조건. 동시에 currentContext를 write하지 않는다.

`src/services/tools/toolOrchestration.ts:19–81`

본문 장 · 심층 독해

A18 · 공통 permission-and-call pipeline

schema가 맞는 입력을 곧바로 실행하지 않고 hook·validation·authority·call 순서를 통과시킨다.

보존 조건. 사용자가 승인한 input과 call input의 identity를 추적한다.

`src/services/tools/toolExecution.ts:599–720`

본문 장 · 심층 독해

A19 · Rule source aggregation과 behavior precedence

설정 source 순서와 allow/deny/ask 평가 순서는 같은 개념이 아니다.

보존 조건. rule에는 source가 남는다.

`src/utils/permissions/permissions.ts:109–131`

본문 장

A20 · Tool-level deny/ask short circuit

도구 전체 rule은 구체 input permission보다 앞서 빠른 terminal decision을 만든다.

보존 조건. deny는 하위 allow가 조용히 덮지 않는다.

`src/utils/permissions/permissions.ts:1070–1120`

본문 장

A21 · Compound command decision 합성

복합 shell 명령은 subcommand별 결과를 계산하고 전체 decision으로 접는다.

보존 조건. 부분 allow로 compound command를 실행하지 않는다.

`src/tools/BashTool/bashPermissions.ts:2430–2485`

본문 장

A42 · Hook fan-out과 permission decision 합성

일치하는 여러 hook을 병렬 실행하되 각 결과를 progress·context·input mutation·permission lattice로 순차 투영한다.

보존 조건. 신뢰되지 않은 workspace의 hook은 실행하지 않는다.

`src/utils/hooks.ts:1952–2075`

본문 장

A43 · 설정과 policy를 sandbox runtime capability로 투영

permission rule과 sandbox setting의 서로 다른 path 문법을 해석하고 network·read·write capability를 runtime config로 낮춘다.

보존 조건. managed-only가 켜지면 낮은 authority source가 allow 범위를 넓히지 못한다.

`src/utils/sandbox/sandbox-adapter.ts:172–260`

본문 장

A22 · Read가 edit precondition을 만든다

FileRead는 출력만 반환하지 않고 이후 Edit가 검증할 file observation state를 남긴다.

보존 조건. partial read 범위를 구분한다.

`src/tools/FileReadTool/FileReadTool.ts:1020–1040`

본문 장

A23 · FileEdit optimistic validation과 commit window

stale check 뒤 write까지 JS event loop에 제어를 넘기지 않는 좁은 동기 구간을 만든다.

보존 조건. check와 synchronous write 사이 내부 await가 없다.

`src/tools/FileEditTool/FileEditTool.ts:427–491`

본문 장 · 심층 독해

A24 · Tool contract characterization

리팩터링 전에 각 tool의 관찰 가능한 contract를 자동 추출·고정한다.

보존 조건. 실제 effect는 fake adapter로 격리한다.

`src/Tool.ts:743–791`

본문 장

A25 · Leaf-to-root conversation chain reconstruction

resume은 최신 leaf에서 parentUuid를 따라가며 root-to-leaf history를 만든다.

보존 조건. 같은 UUID를 두 번 방문하지 않는다.

`src/utils/sessionStorage.ts:2069–2117`

본문 장 · 심층 독해

A26 · 병렬 tool result orphan recovery

single-parent chain이 잃은 sibling assistant와 tool_result를 message.id와 parentUuid index로 되찾는다.

보존 조건. 같은 recovered UUID를 중복 삽입하지 않는다.

`src/utils/sessionStorage.ts:2118–2205`

본문 장

A27 · Resume consistency telemetry

저장 당시 messageCount와 replay 위치를 비교해 round-trip drift를 측정한다.

보존 조건. 가장 최근 checkpoint 하나를 사용한다.

`src/utils/sessionStorage.ts:2208–2243`

본문 장

A28 · Cooperative cancellation propagation

하나의 abort intent를 provider, tool, child task가 각 safe point에서 관찰한다.

보존 조건. abort 후 새 effect를 시작하지 않는다.

`src/services/tools/toolExecution.ts:413–468`

본문 장

A29 · Post-compaction context restoration

요약 뒤에도 최근 file, plan, skills, async agents를 typed attachment로 재주입한다.

보존 조건. preserved content를 중복 주입하지 않는다.

`src/services/compact/compact.ts:1399–1448`

본문 장

A40 · Session memory extraction trigger

token growth, tool-call 수, 자연스러운 turn 경계를 함께 사용해 비싼 memory extraction의 시점을 결정한다.

보존 조건. token growth threshold는 두 trigger 경로 모두에서 반드시 필요하다.

`src/services/SessionMemory/sessionMemory.ts:134–179`

본문 장

A41 · Capability가 축소된 memory extraction agent

parent context를 복제하지 않고 별도 context와 "정확한 memory file 하나의 Edit만" 허용하는 canUseTool로 보조 agent 권한을 줄인다.

보존 조건. subagent와 teammate query에서는 자동 extraction을 실행하지 않는다.

`src/services/SessionMemory/sessionMemory.ts:272–337`

본문 장 · 심층 독해

A30 · Child agent runtime spawn과 event projection

AgentTool은 parent context를 투영해 별도의 query loop를 열고 bounded result를 parent로 돌려준다.

보존 조건. child authority가 parent보다 조용히 넓어지지 않는다.

`src/tools/AgentTool/runAgent.ts:500–770`

본문 장 · 심층 독해

A31 · Incomplete child tool call filtering

child 결과를 합류할 때 unresolved tool_use를 그대로 parent context에 노출하지 않는다.

보존 조건. resolved pair는 유지한다.

`src/tools/AgentTool/runAgent.ts:866–905`

본문 장

A32 · MCP server name normalization

외부 이름을 API-safe identity로 바꾸되 delimiter 충돌을 줄인다.

보존 조건. 결과는 API name pattern을 만족한다.

`src/services/mcp/normalization.ts:1–23`

본문 장

A33 · Background plugin installation queue

plugin install은 UI blocking을 피하면서 marketplace 상태와 실패를 조정한다.

보존 조건. 한 plugin의 실패가 상태를 영원히 installing으로 남기지 않는다.

`src/services/plugins/PluginInstallationManager.ts:60–120`

본문 장

A44 · WebSocket close code별 bounded reconnect

연결 종료를 permanent, session-not-found transient, ordinary transient로 분류하고 서로 다른 retry budget으로 terminal 여부를 결정한다.

보존 조건. permanent close는 reconnect하지 않는다.

`src/remote/SessionsWebSocket.ts:208–300`

본문 장 · 심층 독해

A34 · Settings precedence resolution

값 병합은 source authority와 provenance를 함께 다뤄야 한다.

보존 조건. later override 규칙이 일관된다.

`src/utils/settings/constants.ts:3–22`

본문 장

A35 · Remote managed settings checksum과 polling

원격 policy는 fetch·auth·checksum·disk cache·background poll을 가진 control plane이다.

보존 조건. 동일 settings는 동일 checksum을 낸다.

`src/services/remoteManagedSettings/index.ts:105–155`

본문 장

A36 · Idempotent migration

durable 설정은 새 코드가 오래된 key와 model alias를 반복 실행 안전하게 변환해야 한다.

보존 조건. 반복 실행 결과가 같다.

`src/migrations/migrateLegacyOpusToCurrent.ts:1–57`

본문 장

A37 · Analytics fan-out과 sink isolation

event producer는 sink 목록에 기록하고 sink 실패를 product flow와 분리한다.

보존 조건. telemetry 실패가 core effect를 바꾸지 않는다.

`src/services/analytics/index.ts:72–165`

본문 장

A38 · Batched exporter flush

event를 buffer하고 size/time/shutdown 경계에서 전송해 latency와 신뢰성을 절충한다.

보존 조건. 동시에 같은 record를 두 batch가 소유하지 않는다.

`src/services/analytics/datadog.ts:85–130`

본문 장

부록 B

14개 repository domain

coverage는 모든 파일을 같은 깊이로 다룬다는 뜻이 아니라 중요한 책임 영역이 책의 탐색 지도에서 사라지지 않았다는 뜻이다.

D01

source truth and reconstruction

분석 대상의 provenance와 복원 변형이 이후 주장 범위를 어떻게 제한하는가?

부모 evidence 주제. 공식 개발 저장소가 아니라 복원 산출물이다 · 복원용 TypeScript 설정이 품질 인증은 아니다 · 복사본 변환과 반복 stub 생성 · 원본을 직접 고치는 prepare-src

---

D02

process composition and surfaces

process route, composition root, interactive/headless shell은 상태와 dependency lifetime을 어떻게 나누는가?

부모 evidence 주제. 우선순위 기반 CLI route resolution · Dynamic import가 architecture boundary 역할을 한다 · ask()의 one-shot engine adapter · print loop의 turn 간 상태 handoff

---

D03

interactive ui and input

React/Ink render, synchronous control state, stream projection, keyboard authority가 어떻게 합성되는가?

부모 evidence 주제. QueryGuard는 render state와 query lifecycle을 분리한다 · messagesRef와 React state는 같은 값의 서로 다른 시간 표현이다 · onQueryEvent는 event를 그대로 그리지 않고 보존 정책에 따라 축약한다 · useInput의 isActive와 propagation은 키보드 권한의 일부다 · REPL 안의 네 가지 상태 표현

D04

input context and message model

local input과 instruction/context가 canonical, provider, UI/SDK representation으로 어떻게 변환되는가?

부모 evidence 주제. 사용자 입력 dispatch · System/User/System context 조립 · Discriminated union과 runtime schema는 서로 다른 보증을 맡는다 · API message normalization · 세 가지 message representation

---

D05

turn and generator protocol

한 turn의 resumable execution, event, terminal, tool pairing은 어떤 protocol로 닫히는가?

부모 evidence 주제. yield는 여러 번 반환하는 문법이 아니다 · query()의 event와 return은 다른 채널이다 · queryLoop 상태 전이 · tool-use/tool-result pairing

---

D06

model transport and stream reconstruction

provider partial event가 canonical message, usage, stop reason과 error/retry 판단으로 어떻게 재구성되는가?

부모 evidence 주제. queryModelWithStreaming은 provider adapter를 감싸는 protocol 입구다 · 부분 content block의 typed reconstruction · message_delta는 이미 yield된 message의 마지막 metadata를 정산한다 · stream idle watchdog는 중단과 정상 완료를 구분한다 · Provider event와 내부 상태의 합류 시점

D07

tool runtime and concurrency

model proposal이 typed Tool call과 deterministic concurrent result로 어떻게 바뀌는가?

부모 evidence 주제. Tool generic은 입력과 출력 관계를 보존한다 · Generic factory의 default 합성 · 입력별 concurrency batch partition · 병렬 결과의 deterministic context merge · 공통 permission-and-call pipeline

---

D08

authority hooks and sandbox

rule, hook, user decision, managed policy, sandbox가 최종 effect authority를 어떻게 제한하고 합성하는가?

부모 evidence 주제. Rule source aggregation과 behavior precedence · Tool-level deny/ask short circuit · Compound command decision 합성 · Hook event 이름은 단순 notification이 아니라 서로 다른 제어권을 뜻한다 · Hook fan-out과 permission decision 합성 · HTTP hook는 URL뿐 아니라 DNS·proxy·header secret 경계를 함께 검증한다 · 설정과 policy를 sandbox runtime capability로 투영 · 외부 제어가 effect를 바꾸는 네 경계

D09

file effect and optimistic commit

Read observation이 Edit precondition과 commit window를 어떻게 제한하는가?

부모 evidence 주제. Read가 edit precondition을 만든다 · FileEdit optimistic validation과 commit window · checkPermissions와 call은 서로 다른 시점의 world를 본다 · 사례: config.ts edit의 결정 장부

D10

durability compaction and memory

journal, DAG replay, compaction, cancellation, session memory가 서로 다른 truth와 cutoff를 어떻게 조정하는가?

부모 evidence 주제. Transcript write는 chain identity를 남긴다 · Leaf-to-root conversation chain reconstruction · 병렬 tool result orphan recovery · Cooperative cancellation propagation · Post-compaction context restoration · Session memory extraction trigger · Capability가 축소된 memory extraction agent · 네 가지 기억 표현의 권위와 손실 · Compaction과 memory extraction이 겹치는 경우

D11

agent task and nested runtime

child query, task state, sidechain, cancellation과 parent projection이 어떻게 settlement되는가?

부모 evidence 주제. Task status는 lifecycle algebra다 · Child agent runtime spawn과 event projection · Incomplete child tool call filtering · Parent·Task·Child truth

---

D12

extensions remote and bridge

MCP, plugin, skill, remote/bridge가 local contract와 authority로 들어오며 transport failure를 어떻게 다루는가?

부모 evidence 주제. MCP server name normalization · MCP capability가 Tool이 되기까지 · Background plugin installation queue · Skill은 코드보다 가볍지만 authority와 무관하지 않다 · WebSocket close code별 bounded reconnect · SDK message adapter는 wire representation의 정보 손실을 명시한다 · Remote surface의 네 protocol을 분리한다 · remote prompt가 local effect를 시작하는 경계

D13

configuration variability and migration

build gate, runtime flag, settings precedence, remote policy, migration이 제품 variant를 어떻게 만든다?

부모 evidence 주제. 변형 메커니즘의 시간축 · Settings precedence resolution · Remote managed settings checksum과 polling · Idempotent migration · Build-time gate와 runtime flag를 한 표에 넣되 같은 것으로 부르지 않는다

D14

operations observability and evolution

retry, telemetry, batching, recovery observation을 구조 개선의 검증 oracle로 어떻게 사용하는가?

부모 evidence 주제. API retry와 backoff · Resume consistency telemetry · Analytics fan-out과 sink isolation · Batched exporter flush · 현재 구조의 리팩터링 우선순위 · 새 capability 도입의 전체 decision path

---

부록 C

source index

정의 위치에서 본문 해설과 알고리즘으로 이동한다. 파일 이름만으로 ownership이나 활성 상태를 판정하지 않는다.

---

scripts/build.mjs

- A01 복사본 변환과 반복 stub 생성 · 52-117행

scripts/prepare-src.mjs

- A02 원본을 직접 고치는 prepare-src · 36-76행

src/cli/print.ts

- A05 print loop의 turn 간 상태 handoff · 2140-2188행

src/entrypoints/cli.tsx

- A03 우선순위 기반 CLI route resolution · 28-93행

src/migrations/migrateLegacyOpusToCurrent.ts

- A36 Idempotent migration · 1-57행

src/query.ts

- A10 queryLoop 상태 전이 · 252-321행

src/QueryEngine.ts

- A04 ask()의 one-shot engine adapter · 1249-1295행
- A06 state lifetime classification · 184-207행

---

src/remote/SessionsWebSocket.ts

- A44 WebSocket close code별 bounded reconnect · 208-300행

---

src/services/analytics/datadog.ts

- A38 Batched exporter flush · 85-130행

---

src/services/analytics/index.ts

- A37 Analytics fan-out과 sink isolation · 72-165행

---

src/services/api/claude.ts

- A39 부분 content block의 typed reconstruction · 1940-2054행

---

src/services/api/withRetry.ts

- A12 API retry와 backoff · 170-220행

---

src/services/compact/compact.ts

- A13 Compaction as lossy migration · 596-742행
- A29 Post-compaction context restoration · 1399-1448행

---

src/services/mcp/normalization.ts

- A32 MCP server name normalization · 1-23행

---

src/services/plugins/PluginInstallationManager.ts

- A33 Background plugin installation queue · 60-120행

---

src/services/remoteManagedSettings/index.ts

- A35 Remote managed settings checksum과 polling · 105-155행

---

src/services/SessionMemory/sessionMemory.ts

- A40 Session memory extraction trigger · 134-179행
- A41 Capability가 축소된 memory extraction agent · 272-337행

---

src/services/tools/toolExecution.ts

- A11 tool-use/tool-result pairing · 337-410행
- A18 공통 permission-and-call pipeline · 599-720행
- A28 Cooperative cancellation propagation · 413-468행

---

src/services/tools/toolOrchestration.ts

- A16 입력별 concurrency batch partition · 84-116행
- A17 병렬 결과의 deterministic context merge · 19-81행

---

src/Tool.ts

- A15 Generic factory의 default 합성 · 721-791행
- A24 Tool contract characterization · 743-791행

---

src/tools/AgentTool/runAgent.ts

- A30 Child agent runtime spawn과 event projection · 500-770행
- A31 Incomplete child tool call filtering · 866-905행

---

src/tools/BashTool/bashPermissions.ts

- A21 Compound command decision 합성 · 2430-2485행

---

src/tools/FileEditTool/FileEditTool.ts

- A23 FileEdit optimistic validation과 commit window · 427-491행

---

src/tools/FileReadTool/FileReadTool.ts

- A22 Read가 edit precondition을 만든다 · 1020-1040행

---

src/utils/hooks.ts

- A42 Hook fan-out과 permission decision 합성 · 1952-2075행

---

src/utils/messages.ts

- A09 API message normalization · 1989-2028행
- A14 Message normalization lookup 구축 · 1170-1215행

---

src/utils/permissions/permissions.ts

- A19 Rule source aggregation과 behavior precedence · 109-131행
- A20 Tool-level deny/ask short circuit · 1070-1120행

---

src/utils/processUserInput/processUserInput.ts

- A07 사용자 입력 dispatch · 515-588행

---

src/utils/queryContext.ts

- A08 System/User/System context 조립 · 44-87행

---

src/utils/sandbox/sandbox-adapter.ts

- A43 설정과 policy를 sandbox runtime capability로 투영 · 172-260행

---

src/utils/sessionStorage.ts

- A25 Leaf-to-root conversation chain reconstruction · 2069-2117행
- A26 병렬 tool result orphan recovery · 2118-2205행
- A27 Resume consistency telemetry · 2208-2243행

---

src/utils/settings/constants.ts

- A34 Settings precedence resolution · 3-22행

부록 D

용어집

canonical state	다음 결정과 지속성의 기준이 되는 정규화된 상태. UI에 먼저 보인 partial projection과 구분한다.
composition	domain 동작을 직접 소유하지 않고 구체 구현·설정·adapter를 실행 표면으로 조립하는 경계.
effect	파일·process·network·외부 tool처럼 함수 밖 세계의 상태를 바꾸거나 관찰하는 작용.
idempotency	같은 identity의 요청이 재시도되어도 effect가 중복 정산되지 않게 하는 성질.
projection	event나 canonical state에서 특정 소비자 목적에 맞게 만든 읽기 표현.
referential identity	두 참조가 같은 mutable object를 가리키는 성질. late metadata가 기존 소비자에게 보이는지 결정한다.
suspension point	generator 실행 프레임을 보존한 채 iterator result를 내보내고 다음 resume를 기다리는 지점.

tagged union	명시적 discriminant로 가능한 variant와 각 variant의 payload를 결합한 자료형.
terminal reason	더 이상 같은 실행을 계속하지 않는 이유. 성공·중단·실패·budget 종료를 구분해야 한다.
transition invariant	한 상태에서 다음 상태로 이동할 때 반드시 함께 성립해야 하는 조건.
authority	어떤 effect를 허용하거나 정산되었다고 판정할 권한. 코드 존재나 route 선택과 동일하지 않다.
provenance	값·설정·결정이 어디서 왔으며 어떤 변환을 거쳤는지 추적하는 정보.

부록 E

요구사항 원장 T01–T33

Orca 이전 원문 transcript 전체는 현재 작업공간에서 이용할 수 없다. 아래 T01-T24는 보존된 감사 문서와 산출물에서 복원한 sanitized projection이고 T25-T33은 후속 dispatch 요구에서 보존했다. 따라서 이 원장은 확인 가능한 요구를 설명하지만 완전한 대화 복구를 증명하지 않는다.

ID	요구	상태·위치	한계
T01	저장소와 영상의 독해 관점을 참고하되 언어 메커니즘·프로그래밍 표현·채택 구조·배경지식을 연결한 방대한 학습 자료를 만든다.	**active** front-method · chapter-2 · chapter-4 · appendix-research	영상은 결정적 코드를 설명하는 접근의 참고일 뿐 source truth나 전체 분류 체계가 아니다.
T02	증거 목록보다 저장소를 성립시키는 결정적 구현, 설계·프로그래밍·운영 노력과 리팩터링 압력을 중심에 둔다.	**active** parts-I-V · conclusion	UI와 generator는 여러 큰 축 중 일부이며 보고서 전체를 대체하지 않는다.
T03	원격 탐색용 일회성 도구가 아니라 작업공간의 고정 clone을 분석한다.	**active** front-boundary · manifest.parentInputs	shallow·복원 snapshot이므로 공식 history를 말하지 않는다.

ID	요구	상태·위치	한계
T04	Loop Architecture와 knowledge/reference 재사용 체계를 무시하지 않고 필요한 이론을 연결하되 실행 흐름은 필요할 때만 쓴다.	**active** front-boundary · deep-walkthroughs · manifest.operationBinding	목표와 직접 관련된 재사용 항목만 유지한다.
T05	스터디 구성원과 함께 코드를 읽는 분석 에이전트의 목소리와 저장소 책임 분포에 맞는 편집 구조를 사용한다.	**active** entire-book · research/codebase-study-methods.md	인격적 화자는 증거 경계를 흐리지 않는다.
T06	코드·도식·표·첨부를 사용해 언어 기능 뒤 런타임 메커니즘을 직관적이고 상세하게 설명한다.	**active** six vector diagrams · deep code walkthroughs · attachments	도식은 외부 저작물을 복제하지 않고 관찰된 관계를 새로 그린다.
T07	시간순 요구와 작업 맥락을 보존하고 현재 구현·설계·운영을 구분해 문제와 개선을 설명한다.	**active** current-state-dashboard · requirement-ledger · conclusion	복원되지 않은 원 대화는 UNKNOWN으로 남긴다.
T08	프로젝트 알고리즘과 결정적 보장을 적절한 항목에서 상세히 설명한다.	**active** 10 deep walkthroughs · appendix-algorithms	알고리즘을 형식 증명으로 주장하지 않는다.
T09	디자인 상자가 글을 잘게 끊지 않도록 문장 흐름으로 자연스럽게 읽히게 한다.	**active** natural-flow CSS · chapter prose	짧은 비교와 trace에만 제한적으로 표를 쓴다.
T10	프로젝트 이해, 코드 수준 통찰, 결정적 시스템 성질을 어떻게 보장하는지가 목적이어야 한다.	**active** purpose · all chapter theses · conclusion	보장은 source에 관찰된 invariant와 실패 경계의 설명이며 production 증명이 아니다.
T11	PDF 표현 방식을 조사하고, 한 문단에서 성급히 끝내지 않으며 논리를 충분히 완성한다.	**active** research-adoption · multi-stage walkthrough	분량은 결과이며 글꼴 축소로 맞추지 않는다.
T12	yield를 여러 번 반환으로 설명하지 말고 suspension point, 보존된 frame, terminal return, abrupt throw, iterator result와 query/UI 의미를 설명한다.	**active** chapter-4 language mechanism interlude · glossary-suspension-point	일반 문법 설명은 실제 query/stream code로 되돌아온다.
T13	yield 설명을 순차 예제와 상세 이론으로 직관적으로 구성한다.	**active** chapter-4 generator interlude	실행 흐름은 suspension 의미를 밝히는 범위에서만 나열한다.
T14	한정된 보고서 틀을 버리고 대상에 맞는 자유로운 구조로 전면 개편한다.	**active** one-volume five-part architecture · variable walkthrough lengths	자유로운 구조에도 source/status trace는 유지한다.
T15	도식을 번호 나열로 끝내지 말고 관계를 산문으로 설명하며 관련 이론 분포를 탐색한다.	**active** diagram captions and following prose · algorithm theory	도식 자체가 source 근거를 대체하지 않는다.
T16	자기소개서 작성 루프의 표현 자유와 근거→해석→의미 연결법만 재사용한다.	**active** prose argument structure	해당 업무의 schema·score·도메인은 가져오지 않는다.

ID	요구	상태·위치	한계
T17	Loop Architecture 맥락과 시간순 요구·추론·작업을 재점검해 누락을 막는다.	**active** requirement reconciliation · operation binding in manifest	operation binding은 승인·출판 authority를 부여하지 않는다.
T18	artifacts 전체가 아니라 현재 목표에 재사용되는 책임과 기능만 컨텍스트에 유지한다.	**active** manifest.parentInputs · non-goals	인접 개선은 실행하지 않고 residual limitation으로 보고한다.
T19	분석 내용을 특정 스크립트·코드·도구에 하드코딩하지 않는다.	**active** content/*.json · generic build-book.mjs	builder에는 일반 렌더링·검증 규칙만 두고 편집 주장은 content에 둔다.
T20	pattern 하나로 의미를 압축하지 말고 단계에 맞는 rubric으로 판정한다.	**active** engineering/validation.md · manifest.validation	단일 합산 점수로 품질을 은폐하지 않는다.
T21	현재 세션 요구를 전부 재점검하고 누락을 보강한 뒤 계속한다.	**active** requirement reconciliation · validation	보존되지 않은 raw transcript까지 완전하다고 주장하지 않는다.
T22	일부 예상 누락이 아니라 대화 전체의 보존된 요구를 다시 파악한다.	**active** requirement ledger appendix	sanitized projection의 범위를 명시한다.
T23	분석 범위를 넓히고 중요한 영역이 빠지지 않도록 14-domain rubric으로 검사한다.	**active** parts-I-V · appendix-domains	넓은 coverage는 모든 파일을 같은 깊이로 설명한다는 뜻이 아니다.
T24	분량 상한을 두지 않고 필요한 상세를 요약·함축으로 대체하지 않는다.	**active** natural-flow book · variable-length deep walkthroughs	중복은 제거하되 논리 단계는 생략하지 않는다.
T25	보고서는 실제 저장소를 함께 분석하는 사용자와 AI 에이전트 사이의 직접 협업 시스템이어야 한다.	**active** reader questions · transfer questions · reading paths	대화체가 증거 상태를 대신하지 않는다.
T26	코드·문서·프로젝트 구조·설계·상태 전이·보장·실패·운영·현재 상태를 연결한 확장 서술을 사용한다.	**active** 10 deep walkthroughs	각 항목을 동일 카드로 강제하지 않고 논증의 필요에 따라 배치한다.
T27	기존 baseline과 evidence 소유권을 보존하고 author/evidence lane을 넘지 않는다.	**active** manifest.parentPreservation · visual-review-request	새 독자판은 기존 6권의 acceptance를 변경하지 않는다.
T28	digest-bound frozen build와 전체 sequence visual review를 분리해 운영한다.	**active** manifest · engineering/visual-review-request.md	이 작업은 시각 verdict나 출판 acceptance를 스스로 닫지 않는다.
T29	현재 존재하는 6개 baseline PDF를 명시하고 보존 상태를 확인한다.	**active** README baseline table · manifest.baselinePdfs	독자판에 6권을 합치거나 재출력하지 않는다.

ID	요구	상태·위치	한계
T30	기존 산출물을 덮어쓰지 않는 별도 PDF 추출 pipeline을 만든다.	**active** book-edition subtree · build-book.mjs	부모 content/build/output은 read-only 입력이다.
T31	가볍게 읽히면서도 프로젝트 현재 상태와 추론 경계가 즉시 보이는 디자인을 사용한다.	**active** dashboard · status rails · single-column prose	가독성을 위해 불확실성을 삭제하지 않는다.
T32	GitHub·블로그·커뮤니티의 실제 프로젝트 분석 방식을 조사하고 채택·기각 근거를 기록해 책에 적용한다.	**active** research/codebase-study-methods.md · content/research-adoption.json · appendix-research	링크 나열이나 외부 도식 복제는 충족으로 보지 않는다.
T33	사용자의 실제 가독성 불만은 이전 on-screen geometry PASS보다 우선하며, 서적형 구조와 서술을 새로 검증한다.	**active** new editorial architecture · visual-review-request	기계 geometry와 새 preview는 독자 가독성 acceptance를 자동으로 닫지 않는다.

부록 F

연구 채택 추적

외부 방법은 이름이 아니라 이 책에서 실제로 바뀐 편집 결정과 validator로 기록한다.

자료	채택	기각	적용·검증
The Architecture of Open Source Applications	각 Part를 책임 질문으로 열고, 파일 카탈로그가 아니라 component interaction과 설계 압력을 서술한다.	모든 장을 동일한 아키텍처 템플릿이나 포괄적 component 목록으로 만드는 방식을 기각한다.	#part-1 · #part-2 · #part-3 · #part-4 · #part-5 · #conclusion **fivePartStructure · chapterQuestionCoverage**

자료	채택	기각	적용·검증
500 Lines or Less	심층 독해마다 독자 질문, 선행 개념, 최소 예제, 실제 source 복귀, 실패 반례, transfer question을 연결한다.	작은 예제가 실제 프로젝트를 완전히 재현하거나 일반화한다고 말하는 방식을 기각한다.	#walkthrough-A03 · #walkthrough-A10 · #walkthrough-A39 · #walkthrough-A17 · #walkthrough-A18 · #walkthrough-A23 · #walkthrough-A25 · #walkthrough-A41 · #walkthrough-A30 · #walkthrough-A44 **deepWalkthroughCount · exampleReturnsToCode**
Code Reading Club	장마다 독자가 source를 보기 전 가설을 세우는 질문과 독해 뒤 다른 변경에 적용하는 transfer question을 둔다.	해설자가 모든 의미를 확정하고 독자를 수동적 소비자로 두는 방식을 기각한다.	#reading-paths · #chapter-2 · #chapter-4 · #chapter-7 · #chapter-11 **readerQuestionsPresent · transferQuestionsPresent**
CodeReading Clubs Resources	10분·60분·심층 경로와 walkthrough의 질문→관찰→적용 순서를 스터디 진행 단위로 제공한다.	repo 전체를 선형으로 읽거나 큰 code dump를 그대로 토론 자료로 삼는 방식을 기각한다.	#reading-paths · #walkthrough-A03 · #walkthrough-A39 · #appendix-sources **readingPathsWithin12 · sourceExcerptBounded**
GitHub code navigation: definitions and references	각 알고리즘에 exact source range를 연결하고 source index에서 같은 경로의 정의·소비 맥락으로 역탐색한다.	검색 hit 수나 파일 이름만으로 ownership과 runtime activation을 단정하는 방식을 기각한다.	#appendix-sources · #algorithm-A01 · #algorithm-A44 **sourceRangesExist · sourceAnchorsResolve**
arc42 overview	front matter의 boundary/context, Part별 building-block 책임, 필요한 곳의 runtime view, 결론의 risk/decision, 부록 glossary를 분리한다.	arc42의 모든 section을 빈칸 없이 채우거나 원 저장소의 공식 아키텍처 문서처럼 보이게 하는 방식을 기각한다.	#front-boundary · #part-1 · #part-5 · #conclusion · #appendix-glossary **hierarchicalViews · statusBoundaries**
C4 model	전체 시스템 지도 뒤 Part별 책임으로 확대하고, turn·permission·recovery에는 동적 인과관계 도식을 사용한다.	모든 C4 level을 의무적으로 그리거나 복원 파일을 공식 deployable container로 단정하는 방식을 기각한다.	#figure-system-topology · #figure-state-ownership · #figure-turn-runtime · #figure-permission-effect · #figure-transcript-recovery · #figure-extension-control-plane **diagramCount>=6 · figureCrossReferences**
Diátaxis	본문은 프로젝트 이해와 이유를 연결하는 explanation으로, 알고리즘·source·glossary·ledger 부록은 빠른 reference로 분리한다.	각 장을 네 문서 유형의 고정 카드로 반복하거나 코드 독해를 API reference로 축소하는 방식을 기각한다.	#chapter-1 · #chapter-13 · #appendix-algorithms · #appendix-sources · #appendix-glossary **explanationReferenceSeparation · allAppendixAnchors**
O'Reilly production style guide	그림 번호·설명·본문 참조, descriptive links, 72자 code display 목표, 자연 흐름 print CSS, 하나의 HTML에서 PDF와 preview를 생성한다.	코드를 작게 축소하거나 페이지 위치만으로 독자를 안내하는 방식, 외부 책의 시각 자산을 복제하는 방식을 기각한다.	#toc · #figure-system-topology · #mechanism-yield-suspension · #appendix-algorithms **internalLinksResolve · codeDisplay<=72 · fontThresholds · noClipping**